



ค่ายโอลิมปิกวิชาการ วิชาคอมพิวเตอร์

อัลกอริทึมการแบ่งแยกและเอาชนะ
(Divide and Conquer algorithm)

สรร รัตนสัญญา

san.r@sci.kmutnb.ac.th

ภาควิชาวิทยาการคอมพิวเตอร์และสารสนเทศ

มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

- ปัญหาและการแก้ปัญหา
- การวิเคราะห์ประสิทธิภาพของอัลกอริทึม
- ความสัมพันธ์เวียนเกิด (Recurrence relation)
- ฟังก์ชันเวียนเกิด (Recursive function)
- การแบ่งแยกและเอาชนะ (Divide and Conquer)
- ตัวอย่างปัญหา
- แบบฝึกหัด

ปัญหาและการแก้ปัญหา

- ปัญหาที่เราพบเจอในชีวิตประจำวัน มีหลากหลายรูปแบบ
- แต่ละปัญหาสามารถแก้ได้ด้วยอัลกอริทึมที่แตกต่างกัน
- เราจึงควรเลือกอัลกอริทึมที่มีประสิทธิภาพและเหมาะสมกับปัญหา
- เพื่อให้สามารถแก้ปัญหานั้นได้อย่างมีประสิทธิภาพ
- เราจึงควรรู้วิธีการเปรียบเทียบประสิทธิภาพของอัลกอริทึม



Analysis of Algorithm Efficiency

1. **Analysis Framework**
2. Asymptotic notations and basic efficiency classes
3. Mathematical analysis of **non-recursive** algorithms
4. Mathematical analysis of **recursive** algorithms

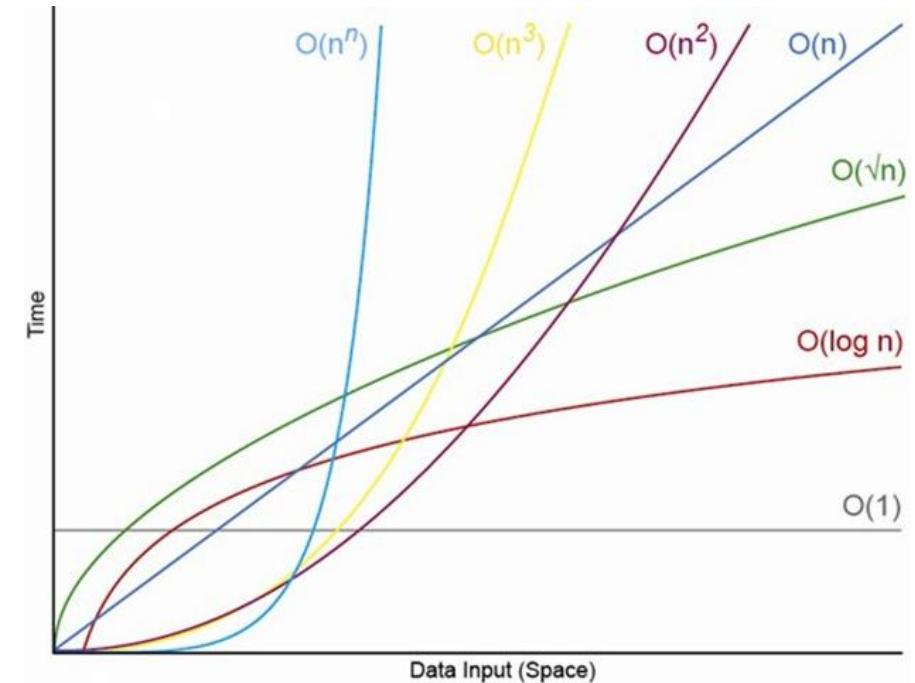
Analysis Framework

- Time efficiency
 - indicates how fast an algorithm in question runs
- Space efficiency
 - deals with the extra space the algorithm requires

Order of Growth

n	$\log_2 n$	n	$n \log_2 n$	n^2	n^3	2^n	$n!$
10	3.3	10^1	$3.3 \cdot 10^1$	10^2	10^3	10^3	$3.6 \cdot 10^6$
10^2	6.6	10^2	$6.6 \cdot 10^2$	10^4	10^6	$1.3 \cdot 10^{30}$	$9.3 \cdot 10^{157}$
10^3	10	10^3	$1.0 \cdot 10^4$	10^6	10^9		
10^4	13	10^4	$1.3 \cdot 10^5$	10^8	10^{12}		
10^5	17	10^5	$1.7 \cdot 10^6$	10^{10}	10^{15}		
10^6	20	10^6	$2.0 \cdot 10^7$	10^{12}	10^{18}		

Table 2.1 Values (some approximate) of several functions important for analysis of algorithms



Best-case, average-case, worst-case

- นอกจาก input size แล้ว จำนวนการทำงานซ้ำของ basic operation อาจขึ้นกับลักษณะของ input ด้วย
 - Worst case: $W(n)$ – maximum over inputs of size n
 - Best case: $B(n)$ – minimum over inputs of size n
 - Average case: $A(n)$ – “average” over inputs of size n
 - Number of times the basic operation will be executed on typical input
 - NOT the average of worst and best case
 - Expected number of basic operations repetitions considered as a random variable under some assumption about the probability distribution of all possible inputs of size n

Sequential Search

Algorithm SequentialSearch(A[0..n], K)

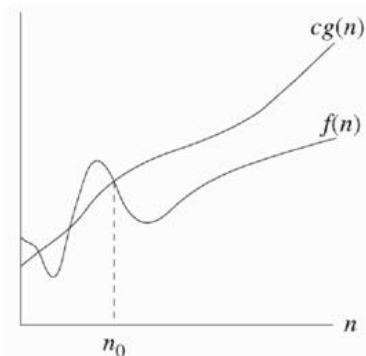
```
A[n] ← K  
i ← 0  
while A[i] ≠ K do  
    i ← i+1  
if i < n return i  
else return -1
```

- *Problem:* กำหนดให้มีชุดข้อมูลใน array จำนวน n ชุด ให้หาชุดข้อมูลที่มีค่าเท่ากับ K
- *Algorithm:* ทำการตรวจสอบข้อมูลที่ละตัวไปเรื่อยๆ ว่ามีตัวใดมีค่าเท่ากับ K จนกว่าจะพบ (*successful search*) หรือจะหมดข้อมูลที่จะทำการค้นหา (*unsuccessful search*)
- Worst case : $C_w(n) = n$
- Best case : $C_b(n) = 1$
- Average case:

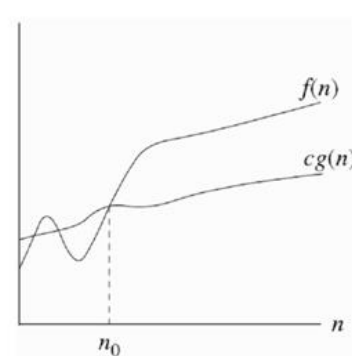
$$C_{avg}(n) = \left[1 \cdot \frac{p}{n} + 2 \cdot \frac{p}{n} + \dots + i \cdot \frac{p}{n} + \dots + n \cdot \frac{p}{n} \right] + n \cdot (1 - p)$$
$$= \frac{p(n+1)}{2} + n(1 - p)$$

Asymptotic growth rate

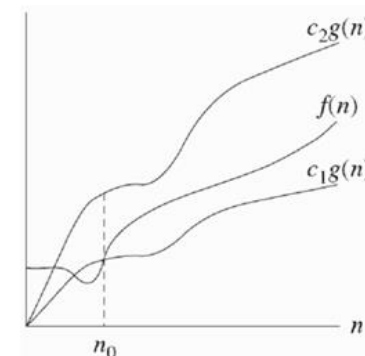
- A way of comparing functions that ignores constant factors and small input sizes
- $O(g(n))$: class of functions $f(n)$ that grow no faster than $g(n)$
- $\Theta(g(n))$: class of functions $f(n)$ that grow at same rate as $g(n)$
- $\Omega(g(n))$: class of functions $f(n)$ that grow at least as fast as $g(n)$



O Big-O Notation



Ω Omega Notation



Θ Theta Notation

Big-oh

$$t(n) \in O(g(n))$$

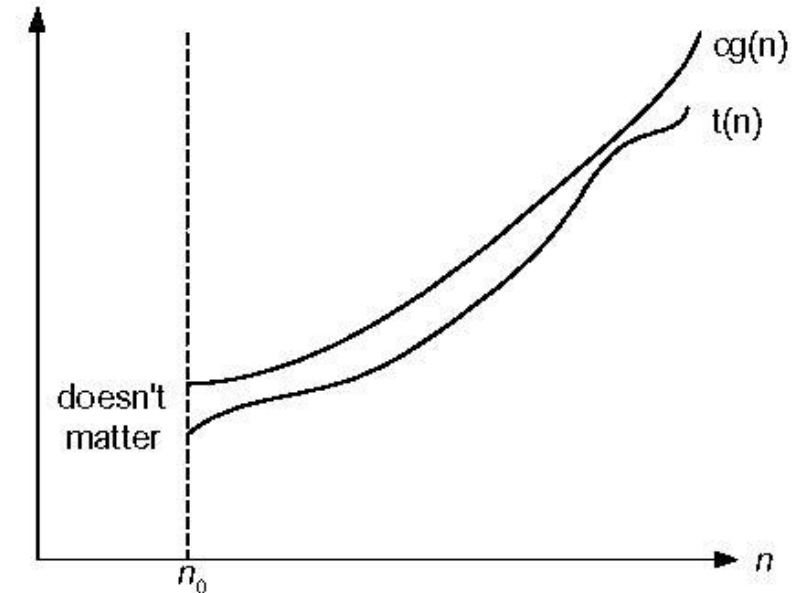
- if $t(n)$ is bounded above by some constant multiple of $g(n)$ for all large n , i.e., if there exist some positive constant c and some nonnegative integer n_0 such that

$$t(n) \leq cg(n) \text{ for all } n \geq n_0.$$

- E.g., $100n + 5 \in O(n^2)$

$$100n+5 \leq 100n+n \text{ (for all } n \geq 5) = 101n \leq 101n^2$$

$$\text{i.e., } c = 101, n_0 = 5$$



Big-omega

$$t(n) \in \Omega(g(n))$$

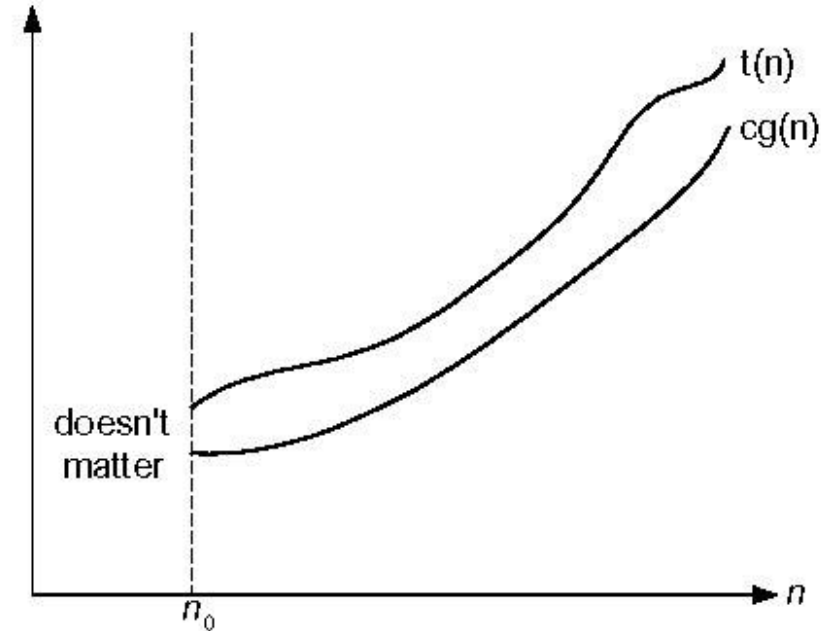
- if $t(n)$ is bounded below by some positive constant multiple of $g(n)$ for all large n , i.e., if there exist some positive constant and some nonnegative integer n_0 such that

$$t(n) \geq cg(n) \text{ for all } n \geq n_0.$$

- E.g., $n^3 \in \Omega(n^2)$

$$n^3 \geq n^2 \quad \text{for } n \geq 0$$

i.e., $c = 1, n_0 = 0$



$$\frac{1}{2}n(n-1) \in \Theta(n^2)$$

$$\mapsto \frac{1}{2}n(n-1) = \frac{1}{2}n^2 - \frac{1}{2}n \leq \frac{1}{2}n^2$$

$$\mapsto \frac{1}{2}n(n-1) = \frac{1}{2}n^2 - \frac{1}{2}n \geq \frac{1}{2}n^2 - \frac{1}{2}n \frac{1}{2}n = \frac{1}{4}n^2$$

$$c_2 = 1/4, c_1 = 1/2, n_0 = 2$$

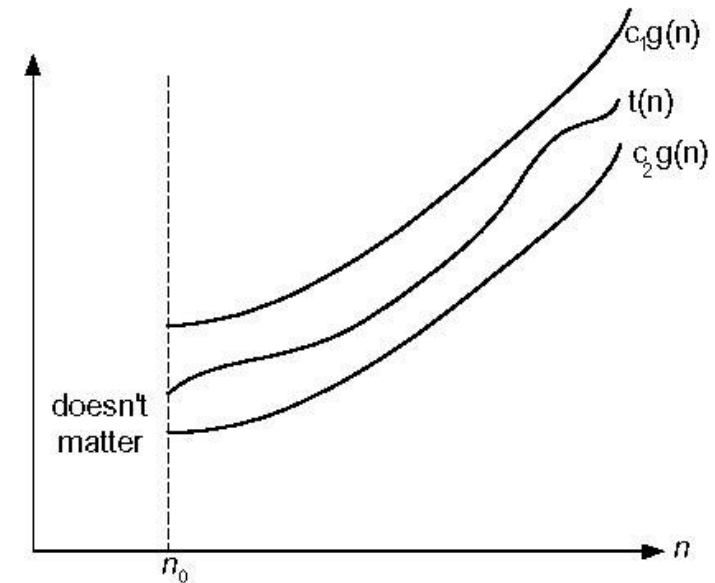
Big-theta

- if $t(n)$ is bounded both above and below by some positive constant multiples of $g(n)$ for all large n , i.e., if there exist some positive constant c_1 and c_2 and some nonnegative integer n_0 such that

$$c_2 g(n) \leq t(n) \leq c_1 g(n) \quad \text{for all } n \geq n_0.$$

- E.g.,

$$c_2 = \frac{1}{4}, c_1 = \frac{1}{2}, n_0 = 2$$



$$\frac{1}{2} n(n-1) \in \Theta(n^2)$$

$$\mapsto \frac{1}{2} n(n-1) = \frac{1}{2} n^2 - \frac{1}{2} n \leq \frac{1}{2} n^2$$

$$\mapsto \frac{1}{2} n(n-1) = \frac{1}{2} n^2 - \frac{1}{2} n \geq \frac{1}{2} n^2 - \frac{1}{2} n \frac{1}{2} n = \frac{1}{4} n^2$$

Basic Asymptotic Efficiency classes

1	constant
$\log n$	logarithmic
n	linear
$n \log n$	$n \log n$
n^2	quadratic
n^3	cubic
2^n	exponential
$n!$	factorial

การใช้ทฤษฎีลิมิตเพื่อเปรียบเทียบอัตราการเติบโต

$$\lim_{n \rightarrow \infty} \frac{t(n)}{g(n)} = \begin{cases} 0 \\ c > 0 \\ \infty \end{cases}$$

อัตราการเติบโตของ $t(n)$ **น้อยกว่า** $g(n)$

อัตราการเติบโตของ $t(n)$ **เท่ากับ** $g(n)$

อัตราการเติบโตของ $t(n)$ **มากกว่า** $g(n)$

Examples:

■ $10n$ vs. $2n^2$

■ $n(n+1)/2$ vs. n^2

L'Hôpital's rule

If

$$\lim_{n \rightarrow \infty} f(n) = \lim_{n \rightarrow \infty} g(n) = \infty$$

The derivatives f' , g' exist,

Then

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \lim_{n \rightarrow \infty} \frac{f'(n)}{g'(n)}$$

Example: $\log n$ vs. n

Mathematical analysis of **non-recursive** algorithms

■ Non-recursive Algorithm Time Efficiency

- ระบุขนาดของข้อมูล (input size) และ basic operation ของอัลกอริทึม
 - Basic operation มักที่อยู่ในลูปชั้นในสุด
- พิจารณาว่าอัลกอริทึมมี worst, average, and best case หรือไม่
 - จำนวนครั้งการทำงานของ basic operation แตกต่างกันเมื่อข้อมูลต่างกัน แม้ขนาดข้อมูลเท่ากัน
- ตั้งสมการ summation ของ basic operation
- แก้สมการเพื่อหา Order of growth

อัลกอริทึม Selection sort

Algorithm SelectionSort($A[0..n-1]$)

```
for i  $\leftarrow$  0 to n-2 do
  min  $\leftarrow$  i
  for j  $\leftarrow$  i+1 to n-1 do
    if  $A[j] < A[\text{min}]$ 
      min  $\leftarrow$  j
  swap  $A[i]$  and  $A[\text{min}]$ 
```

Time Efficiency?

อัลกอริทึม Insertion sort

Algorithm InsertionSort($A[0..n-1]$)

for $i \leftarrow 1$ **to** $n-1$ **do**

$v \leftarrow A[i]$

$j \leftarrow i-1$

while $j \geq 0$ **and** $A[j] > v$ **do**

$A[j+1] \leftarrow A[j]$

$j \leftarrow j-1$

$A[j+1] \leftarrow v$

Time Efficiency?

■ Recursive Algorithm Time Efficiency

- ระบุขนาดของข้อมูล (input size) และ basic operation ของอัลกอริทึม
 - Basic operation มักที่อยู่ในรูปขั้นในสุด
- พิจารณาว่าอัลกอริทึมมี worst, average, and best case หรือไม่
 - จำนวนครั้งการทำงานของ basic operation แตกต่างกันเมื่อข้อมูลต่างกัน แม้ขนาดข้อมูลเท่ากัน
- ตั้งสมการ Recurrence relation ของ basic operation
 - ต้องระบุ **initial condition** ด้วยเสมอ
- แก้สมการเพื่อหา Order of growth

Tower of Hanoi

- การย้าย N disks จากเสา 1 ไปยังเสา 3 โดยมีเสา 2 เป็นเสาพัก ทำได้โดย
 - ย้าย (N-1) disks จากเสา 1 ไปยังเสา 2
 - ย้าย disk แผ่นล่างสุด ไปยังเสา 3
 - ย้าย (N-1) disks จากเสา 2 ไปยังเสา 3
- จำนวนครั้งของการเคลื่อนย้าย
$$H(n) = H(n-1) + 1 + H(n-1), \text{ for } n > 1$$
และ $H(1) = 1$

Solve The Tower of Hanoi recurrence relation

$$\begin{aligned}H_n &= 2H_{n-1} + 1 \\&= 2(2H_{n-2} + 1) + 1 = 2^2H_{n-2} + 2 + 1 \\&= 2^2(2H_{n-3} + 1) + 2 + 1 = 2^3H_{n-3} + 2^2 + 2 + 1 \\&\vdots \\&= 2^{n-1}H_1 + 2^{n-2} + 2^{n-3} + \dots + 2 + 1 \\&= 2^{n-1} + 2^{n-2} + \dots + 2 + 1 \\&= 2^n - 1.\end{aligned}$$

Important recurrence types:

- One (constant) operation reduces problem size by one.

$$T(n) = T(n-1) + c \qquad T(1) = d$$

$$\text{Solution: } T(n) = (n-1)c + d \qquad \textit{linear}$$

- A pass through input reduces problem size by one.

$$T(n) = T(n-1) + cn \qquad T(1) = d$$

$$\text{Solution: } T(n) = [n(n+1)/2 - 1] c + d \qquad \textit{quadratic}$$

- One (constant) operation reduces problem size by half.

$$T(n) = T(n/2) + c \qquad T(1) = d$$

$$\text{Solution: } T(n) = c \lg n + d \qquad \textit{logarithmic}$$

- A pass through input reduces problem size by half.

$$T(n) = 2T(n/2) + cn \qquad T(1) = d$$

$$\text{Solution: } T(n) = cn \lg n + d n \qquad \textit{n log n}$$

Important recurrence types:

- One (constant) operation reduces problem size by one.

$$T(n) = T(n-1) + c \qquad T(1) = d$$

$$\text{Solution: } T(n) = (n-1)c + d \qquad \underline{\text{linear}}$$

- A pass through input reduces problem size by one.

$$T(n) = T(n-1) + cn \qquad T(1) = d$$

$$\text{Solution: } T(n) = [n(n+1)/2 - 1] c + d \qquad \underline{\text{quadratic}}$$

- One (constant) operation reduces problem size by half.

$$T(n) = T(n/2) + c \qquad T(1) = d$$

$$\text{Solution: } T(n) = c \lg n + d \qquad \underline{\text{logarithmic}}$$

- A pass through input reduces problem size by half.

$$T(n) = 2T(n/2) + cn \qquad T(1) = d$$

$$\text{Solution: } T(n) = cn \lg n + d n \qquad \underline{n \log n}$$

■ ข้อใดผิด

Big-Oh Examples

- $n \in O(n) ?$
- $10n \in O(n) ?$
- $n \in O(10n) ?$
- $n \in O(n^2) ?$
- $n^2 \in O(n) ?$
- $10n^2 \in O(n^2) ?$
- $n \lg n \in O(n^2) ?$
- $\ln n \in O(2n) ?$
- $\lg n \in O(n) ?$
- $3n + 4 \in O(n) ?$
- $5n^2 + 10n - 2 \in O(n^3) ? O(n^2) ? O(n) ?$

Big-Oh Examples

- $\sqrt{n} \in O(n) ?$
- $\lg n \in O(2^n) ?$
- $\lg n \in O(n) ?$
- $n \lg n \in O(n) ?$
- $n \lg n \in O(n^2) ?$
- $\sqrt{n} \in O(\lg n) ?$
- $\lg n \in O(\sqrt{n}) ?$
- $n \lg n \in O(n^{\frac{3}{2}}) ?$
- $n^3 + n \lg n + n\sqrt{n} \in O(n \lg n) ?$
- $n^3 + n \lg n + n\sqrt{n} \in O(n^3) ?$
- $n^3 + n \lg n + n\sqrt{n} \in O(n^4) ?$

■ จงหา Big-O ของ code ต่อไปนี้

■ 1.

```
for (i=1; i<n; i=i+2)
    sum++;
```

■ 2.

```
for (i=1; i<n; i++)
    for (j=1; j < n/2; j++)
        sum++;
```

■ 3.

```
for (i=0; i<n; i++)
    for (j = 0; j<i; j++)
        sum++;
```

■ 4.

```
for (i=0; i<n;i++)
    for (j=1; j<n; j=j*2)
        for (k=1; k<n; k=k+2)
            sum++;
```

แบบฝึกหัด 4

- จงเขียนโปรแกรมคูณเลขฐาน 2 $(10101011101010101)_2 \times (10101011101010101)_2$ พร้อมทั้งคำนวณ Big-O โดยให้คูณทีละหลักจากหลังไปหน้า
- จงเขียนโปรแกรมเรียงข้อมูลของ $A[n]$ จากน้อยไปมาก โดย Input คือ จำนวนข้อมูลและข้อมูล โดยเวลาที่ใช้ในการประมวลผลคือ $O(n \log n)$
- จงเขียนโปรแกรมเรียงข้อมูลของ $A[n]$ จากน้อยไปมาก โดย Input คือ จำนวนข้อมูลและข้อมูล โดยเวลาที่ใช้ในการประมวลผลคือ $O(n)$

แบบฝึกหัด 4

- (Program) จงเขียนโปรแกรมคูณเลขฐาน 2 $(10101011101010101)_2 \times (10101011101010101)_2$ พร้อมทั้งคำนวณ Big-O โดยให้ข้อมูลที่ละหลักจากหลังไปหน้า
- (Program) จงเขียนโปรแกรมเรียงข้อมูลของ $A[n]$ จากน้อยไปมาก โดย Input คือ จำนวนข้อมูลและข้อมูล โดยเวลาที่ใช้ในการประมวลผลคือ $O(n \log n)$
- (Program) จงเขียนโปรแกรมเรียงข้อมูลของ $A[n]$ จากน้อยไปมาก โดย Input คือ จำนวนข้อมูลและข้อมูล โดยเวลาที่ใช้ในการประมวลผลคือ $O(n)$

แบบฝึกหัด 5

กำหนดให้เมทริกซ์

$$A = \begin{bmatrix} 2 & 5 & 6 & 3 & 7 \\ 6 & 2 & 8 & 7 & 9 \\ 3 & 1 & 4 & 7 & 8 \\ 3 & 9 & 7 & 8 & 2 \\ 3 & 5 & 6 & 7 & 2 \end{bmatrix} \quad B = \begin{bmatrix} -1 & 5 & 0 & 6 & 7 \\ 6 & 5 & 1 & 4 & 7 \\ 2 & 5 & 8 & 10 & 9 \\ 9 & 6 & 3 & 2 & 5 \\ 1 & 7 & 2 & 5 & 3 \end{bmatrix}$$

จงเขียนโปรแกรมหาผลลัพธ์ $C = A \times B$ พร้อมทั้งคำนวณ Big-O

ความสัมพันธ์เวียนเกิด (Recurrence relation)

- หรือ Recurrence Relations สำหรับอันดับ $a_0, a_1, \dots, a_n$ เป็นสมการที่แสดงความสัมพันธ์ของพจน์ a_n กับพจน์ $a_0, a_1, \dots, a_{n-1}$ ที่เกิดก่อน โดยมีเงื่อนไขเริ่มต้นสำหรับพจน์ $a_0, a_1, \dots, a_{n-1}$ ที่ชัดเจน หรืออาจจะกล่าวได้ คือตัวเลขถัดไปนั้นมีความสัมพันธ์กับตัวเลขก่อนหน้า โดยสามารถนำมาเขียนสมการคณิตศาสตร์ได้

- ตัวอย่าง จงหาความสัมพันธ์เวียนเกิดของอันดับ 5, 8, 11, 14, 17,...

$$a_0 = 5$$

$$a_1 = 8 = 5+3 = a_0+3$$

$$a_2 = 11 = 8+3 = a_1+3$$

$$a_3 = 14 = 11+3 = a_2+3$$

$$a_4 = 17 = 14+3 = a_3+3$$

พิจารณาพบว่า พจน์ถัดไปมีค่า
มากกว่าพจน์ก่อนหน้าอยู่ 3

พิจารณาต่อไปจะได้

$a_n = a_{n-1}+3 \quad ; n \geq 1$ โดยมี $a_0 = 5$	เป็น ความสัมพันธ์เวียนเกิด เป็น เงื่อนไขเริ่มต้น
--	--

- **แฟกทอเรียล** ของจำนวนเต็มไม่ติดลบ n คือ ผลคูณของจำนวนเต็มบวกทั้งหมดที่น้อยกว่าหรือเท่ากับ n เขียนแทนด้วย $n!$
- คำตอบเกิดจากการคูณของจำนวนเต็มบวกชุดหนึ่ง ซึ่งถ้าคำตอบเกิดจากการคูณของจำนวนเต็มบวกตั้งแต่ 1 ถึง n เช่น $1 \times 2 \times 3 \times 4 \times 5$ จำนวนเหล่านี้เราสามารถใช้สัญลักษณ์ แฟกทอเรียล เขียนแทนได้ คือ $5!$

$$\begin{aligned} N! &= N \times (N-1)! \\ &= N \times (N-1) \times (N-2)! \\ &= N \times (N-1) \times (N-2) \times (N-3) \times \dots \times 3 \times 2 \times 1 \end{aligned}$$

โดย $0!$ มีค่า เป็น 1

โดย $1!$ มีค่า เป็น 1

แฟกทอเรียล

- จากนิยามสามารถสรุปการหาคำตอบของ $N!$ ได้เป็น 2 กรณี คือ
 - ถ้า N มีค่าเท่ากับ 0 คำตอบที่ได้คือ $N! = 1$
 - ถ้า N มีค่ามากกว่า 0 คำตอบที่ได้คือ $N! = N \times (N-1)!$
- สำหรับ $5!$ การคำนวณหาค่าแฟกทอเรียล

$$\begin{aligned}
 5! &= 5 \times 4! \\
 &= 5 \times 4 \times 3! \\
 &= 5 \times 4 \times 3 \times 2! \\
 &= 5 \times 4 \times 3 \times 2 \times 1! \\
 &= 5 \times 4 \times 3 \times 2 \times 1 = 120
 \end{aligned}$$

1	ft = 5 * fact(4);	
2	ft = 5 * 4 * fact(3);	
3	ft = 5 * 4 * 3 * fact(2);	
4	ft = 5 * 4 * 3 * 2 * fact(1);	
5	ft = 5 * 4 * 3 * 2 * 1;	ผลลัพธ์คือ 120

แฟกทอเรียล

$$n! = 1 \times 2 \times 3 \dots \times (n-1) \times n = (n-1)! \times n$$

$$a_n = (n-1)! \times n$$

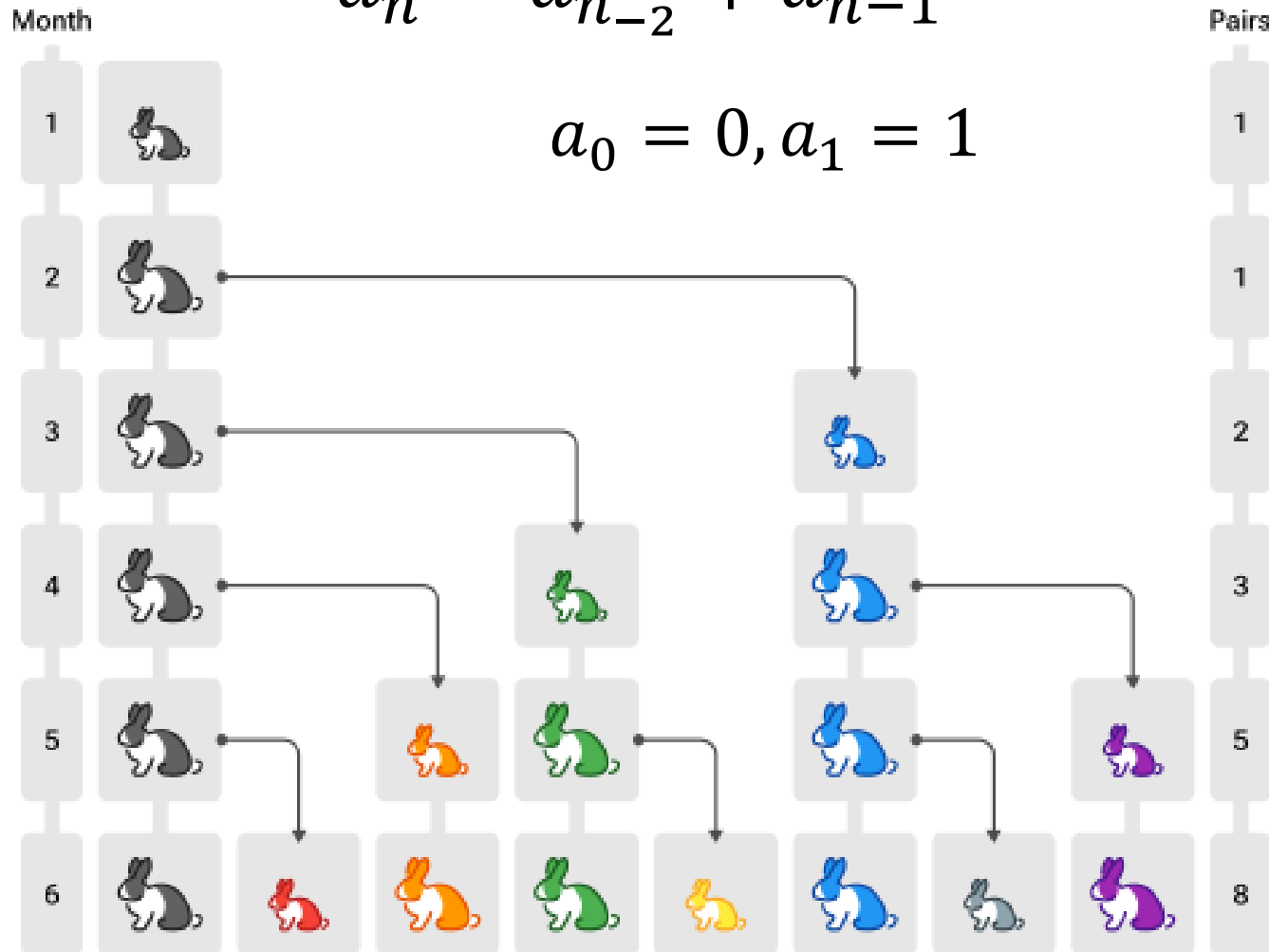
$$= a_{n-1} \times n$$

- ดังนั้น ความสัมพันธ์เวียนเกิดของ $n!$
- คือ $a_n = n a_{n-1} ; n \geq 1$
- เงื่อนไขเริ่มต้น $a_0 = 1$ และ $a_1 = 1$

Fibonacci

$$a_n = a_{n-2} + a_{n-1}$$

$$a_0 = 0, a_1 = 1$$



Golden Ratio



ฟีโบนัชชี

- ลำดับฟีโบนัชชี (Fibonacci Sequence) มีนิยามของความสัมพันธ์ว่า จำนวนถัดไปเท่ากับผลบวกของจำนวนสองจำนวนก่อนหน้า และสองจำนวนแรกก็คือ 0 และ 1 ตามลำดับ หากเขียนให้อยู่ในรูปของสัญลักษณ์ ลำดับ F_n ของฟีโบนัชชี
- สามารถเขียนความสัมพันธ์เวียนเกิดได้ดังนี้

$$F_n = F_{n-1} + F_{n-2}$$

โดยกำหนดค่าเริ่มแรกให้

$$F_0 = 0 \text{ และ } F_1 = 1$$

ฟีโบนัชชี

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55 ,... จากตัวเลขอนุกรมดังกล่าว
สามารถแสดงวิธีการหาค่าเทอมต่างๆ ได้ดังนี้

$$1) F_0 = 0$$

$$2) F_1 = 1$$

$$3) F_n = F_{n-1} + F_{n-2}$$

พจน์ที่ 3 มาจาก พจน์ที่ 1 + พจน์ที่ 2 = $0 + 1 = 1$ โดยเริ่มจากพจน์ที่ 3 เป็นต้นไป

พจน์ที่ 4 มาจาก พจน์ที่ 2 + พจน์ที่ 3 = $1 + 1 = 2$

พจน์ที่ 5 มาจาก พจน์ที่ 3 + พจน์ที่ 4 = $1 + 2 = 3$

ดังนั้น ความสัมพันธ์เวียนเกิด คือ $a_n = a_{n-1} + a_{n-2} ; n \geq 2$

และเงื่อนไขเริ่มต้น $a_0 = 0$ และ $a_1 = 1$

จากนิยามสามารถสรุปการหาคำตอบออกเป็น 2 ทางคือ

ถ้า n มีค่าเป็น 0 หรือ 1 คำตอบที่ได้คือ $F_n = n$

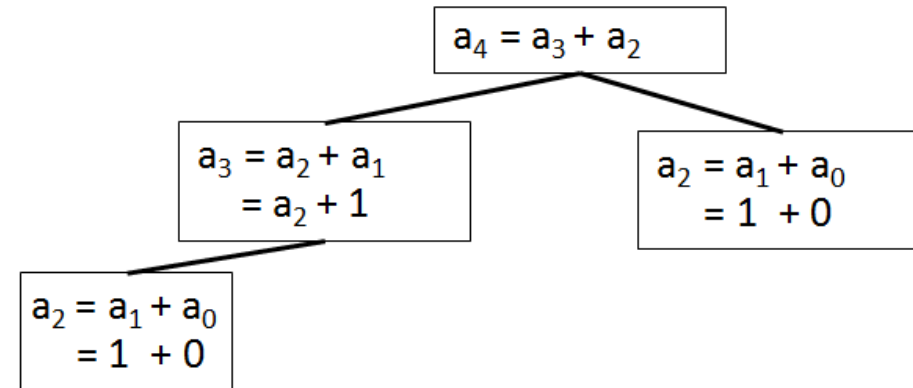
ถ้า n มีค่ามากกว่า 1 คำตอบที่ได้คือ $F_n = F_{n-1} + F_{n-2}$

ฟีโบนัชชี

- จงคำนวณหาค่าอนุกรมฟีโบนัชชีที่ 4
- วิธีคิดแบบต้นไม้แตกกิ่งก้านสาขา

- วิธีคิดแบบปกติ

$$\begin{aligned}
 F(4) &= F(3) + F(2) \\
 &= (F(2) + F(1)) + (F(1) + F(0)) \\
 &= ((F(1) + F(0)) + F(1)) + (F(1) + F(0)) \\
 &= ((1+0) + 1) + (1 + 0) \\
 &= 3
 \end{aligned}$$



ตัวอย่าง 2

- มีเงินฝาก 1,000 บาท อัตราดอกเบี้ย 12% ต่อปี ถ้าฝากแบบดอกเบี้ยทบต้นสิ้นปีที่ n จะมีเงินรวมทั้งหมดเท่าไร

$$\text{สิ้นปีที่ 1 มีเงิน} \quad 1000.00 + (0.12)(1000.00) = 1120.00$$

$$\text{สิ้นปีที่ 2 มีเงิน} \quad 1120.00 + (0.12)(1120.00) = 1254.40$$

$$\text{สิ้นปีที่ 3 มีเงิน} \quad 1254.40 + (0.12)(1254.40) = 1404.92$$

...

$$\text{สิ้นปีที่ } n \text{ มีเงิน} \quad \text{เงินรวมสิ้นปีที่ } n-1 + \text{ดอกเบี้ย}$$

$$\text{ให้ } a_n \text{ เป็นเงินรวมเมื่อสิ้นปีที่ } n$$

$$a_n = (1)(a_{n-1}) + (0.12)(a_{n-1})$$

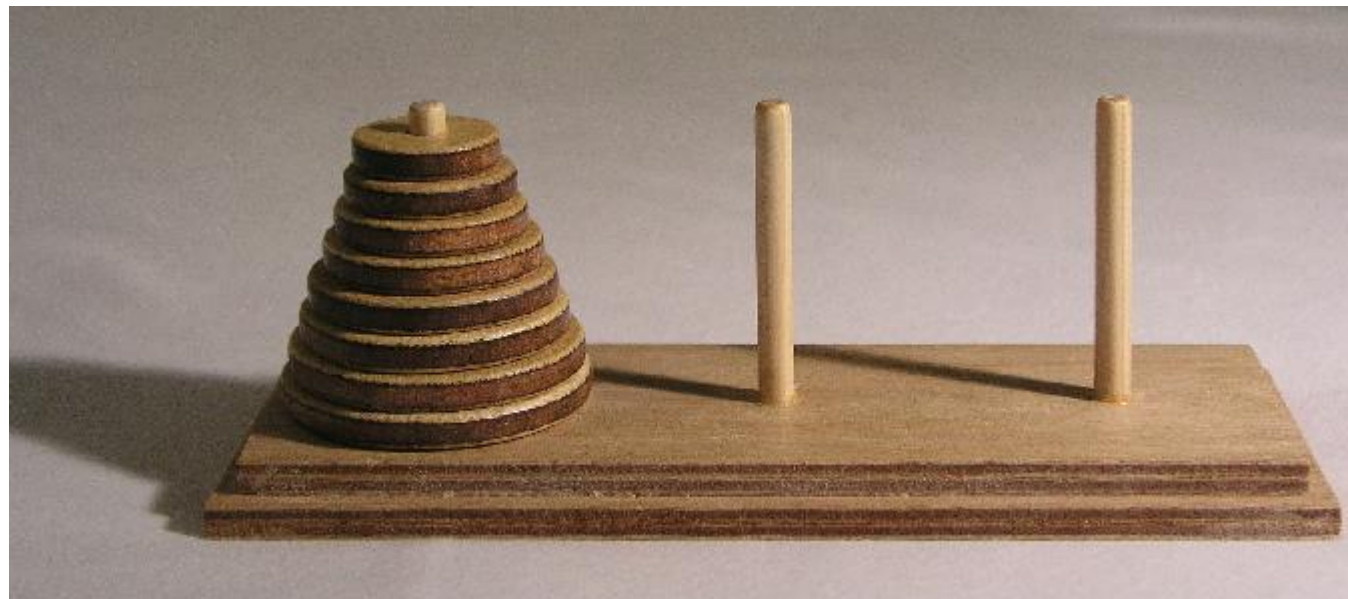
$$a_n = (1.12)(a_{n-1})$$

- ดังนั้น ความสัมพันธ์เวียนเกิด คือ $a_n = (1.12)(a_{n-1}) ; n \geq 1$
และเงื่อนไขเริ่มต้น $a_0 = 1000$

- จงเขียนโปรแกรมแบบ recursive สำหรับปัญหาแต่ละข้อต่อไปนี้
 - คำนวณค่าของ $n!$
 - คำนวณค่าของ a^n
 - หาตัวหารร่วมมากของจำนวนเต็ม a, b, c
 - A Recursive Linear Search Algorithm
 - A Recursive Binary Search Algorithm
 - A Recursive Algorithm for Fibonacci Numbers
 - An Iterative Algorithm for Computing Fibonacci Number

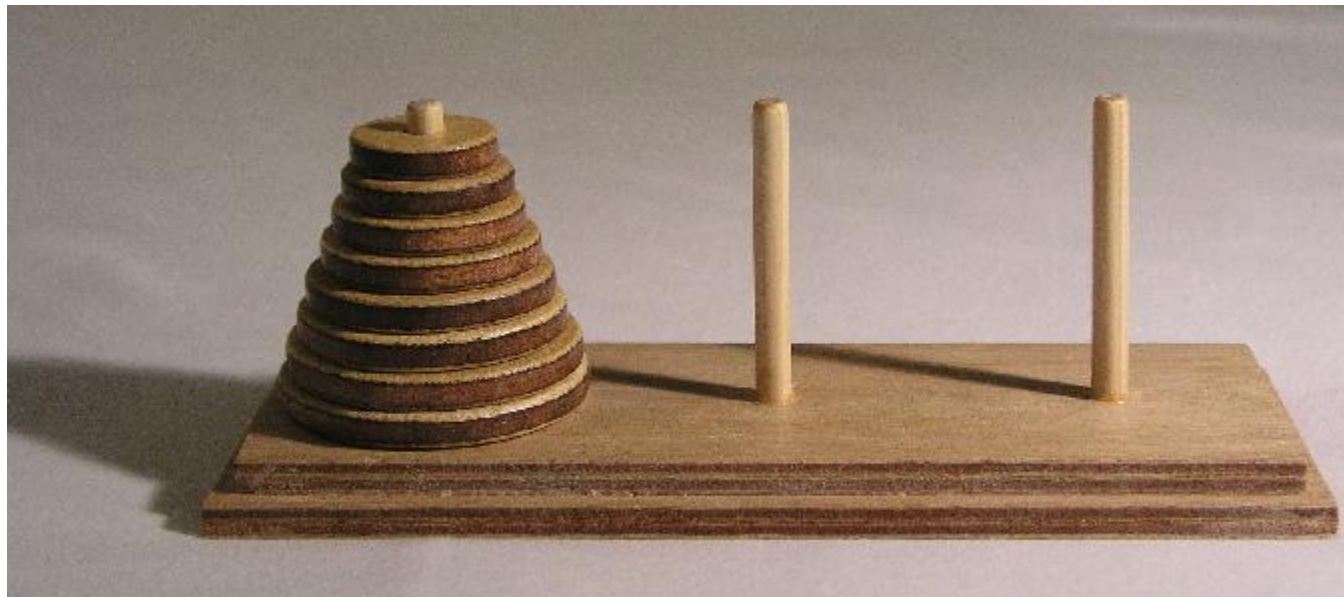
ปริศนาหอคอยฮานอย

- EDOUARD LUCAS คือ นักคณิตศาสตร์ชาวฝรั่งเศส เป็นผู้คิดค้นปริศนาหอคอยฮานอย (The Tower of Hanoi) โดยปริศนาหอคอยฮานอย นั้นจะมีแผ่นจานไม้ 8 แผ่น รัศมีแตกต่างกัน แต่ละแผ่นมีรูตรงกลาง นำมาใส่ไว้ในหลักเป็นกองซ้อน โดยให้แผ่นที่เล็กกว่าทับแผ่นที่ใหญ่กว่า และมีหลักเปล่าสองหลัก ดังรูป

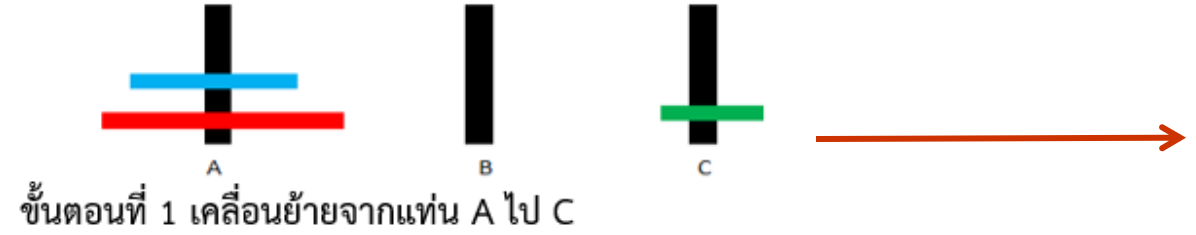


ปริศนาหอคอยฮานอย

- **ปริศนาหอคอยฮานอย** คือ ให้ย้ายแผ่นจานทั้งหมดไปกองไว้ที่หลักเปล่าหลักหนึ่ง โดยมีเงื่อนไขว่า เคลื่อนย้ายได้คราวละแผ่น และต้องนำไปไว้ที่หลักใดหลักหนึ่ง และห้ามแผ่นที่มีขนาดใหญ่กว่าวางทับแผ่นที่มีขนาดเล็กกว่า ต่อไปนี้แสดงขั้นตอนการเคลื่อนย้ายแผ่นจานจำนวน 3 แผ่นจากแผ่น A ไปแผ่น C

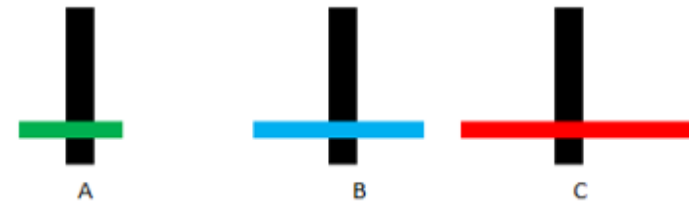
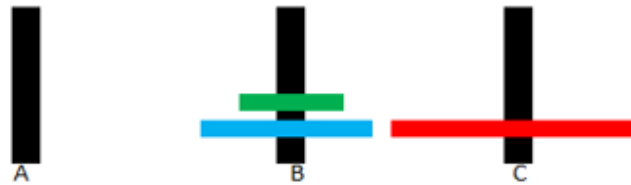


ตัวอย่างการเคลื่อนย้าย



ขั้นตอนที่ 2 เคลื่อนย้ายจากแท่น A ไป B

ขั้นตอนที่ 3 เคลื่อนย้ายจากแท่น C ไป B



ขั้นตอนที่ 4 เคลื่อนย้ายจากแท่น B ไป C

ขั้นตอนที่ 5 เคลื่อนย้ายจากแท่น B ไป A



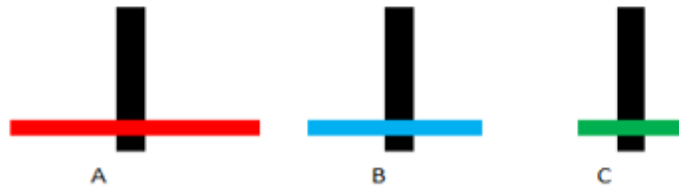
ขั้นตอนที่ 6 เคลื่อนย้ายจากแท่น B ไป C

ขั้นตอนที่ 7 เคลื่อนย้ายจากแท่น A ไป C

ตัวอย่างการเคลื่อนย้าย



ขั้นตอนที่ 1 เคลื่อนย้ายจากแท่น A ไป C



ขั้นตอนที่ 2 เคลื่อนย้ายจากแท่น A ไป C



ขั้นตอนที่ 3 เคลื่อนย้ายจากแท่น C ไป B



ขั้นตอนที่ 4 เคลื่อนย้ายจากแท่น A ไป C



ขั้นตอนที่ 5 เคลื่อนย้ายจากแท่น B ไป A



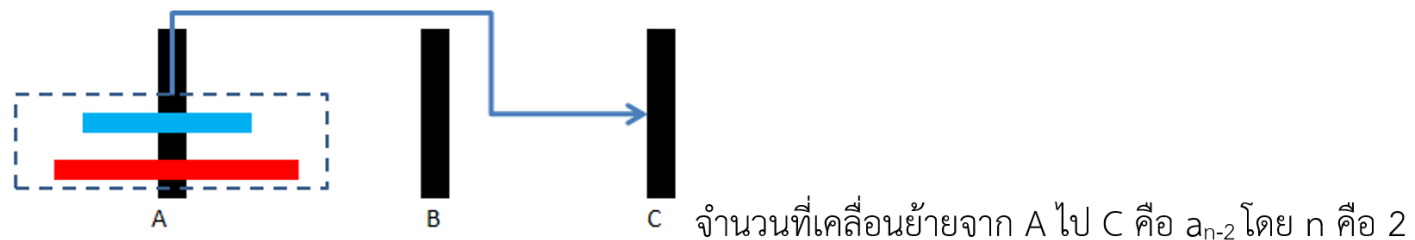
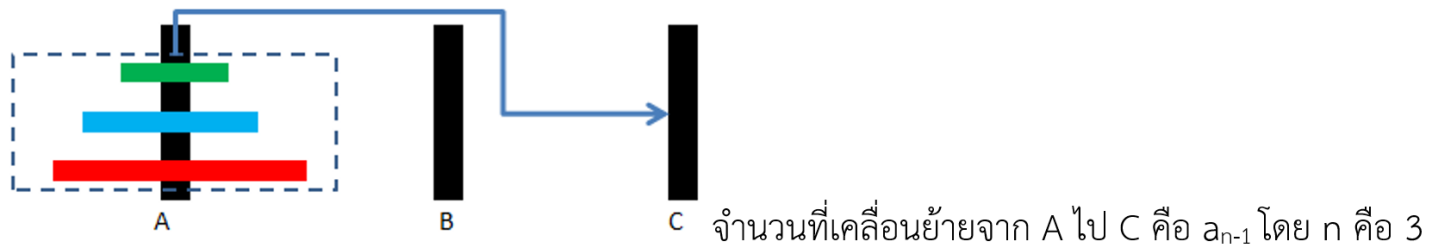
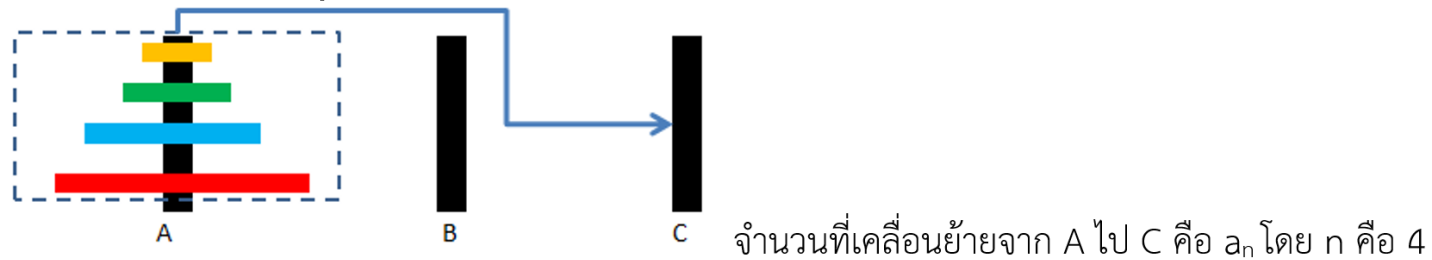
ขั้นตอนที่ 6 เคลื่อนย้ายจากแท่น B ไป C

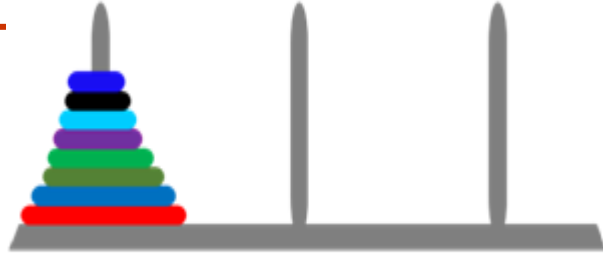


ขั้นตอนที่ 7 เคลื่อนย้ายจากแท่น A ไป C

จำนวนการเคลื่อนย้ายแผ่น

- ให้ a_n เป็นจำนวนครั้งน้อยสุดในการเคลื่อนย้ายแผ่นจาก n แผ่นจากหลักหนึ่งไปยังอีกหลักหนึ่ง





1. ต้องย้ายแผ่นที่เล็กกว่าทั้งหมด $n-1$ แผ่น
ไปยังหลักที่ว่างก่อน จำนวนการเคลื่อนย้าย
คือ a_{n-1} ครั้ง



2. ย้ายแผ่นใหญ่ที่สุดไปยังหลักเป้าหมาย
จำนวนการเคลื่อนย้ายคือ 1 ครั้ง



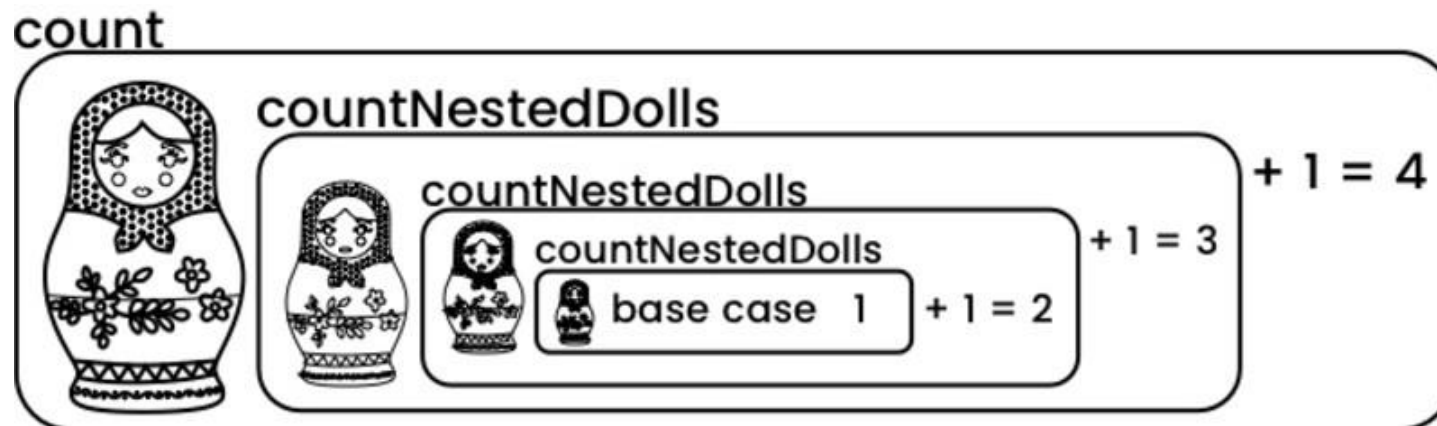
3. ย้ายแผ่นจานทั้งหมดในข้อ 1. ไปยัง
หลักเป้าหมาย จำนวนการเคลื่อนย้าย
คือ a_{n-1} ครั้ง

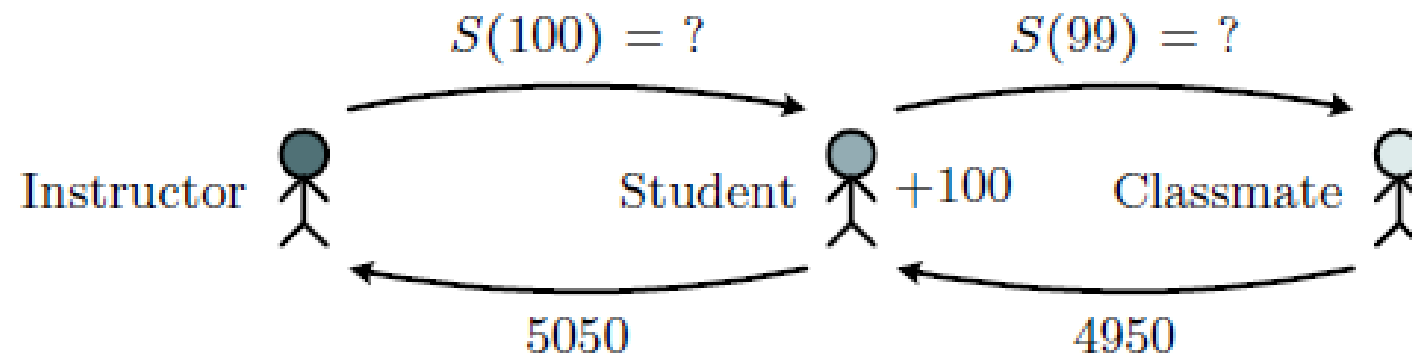


จะได้ $a_n = a_{n-1} + 1 + a_{n-1}$
 $= 2a_{n-1} + 1$

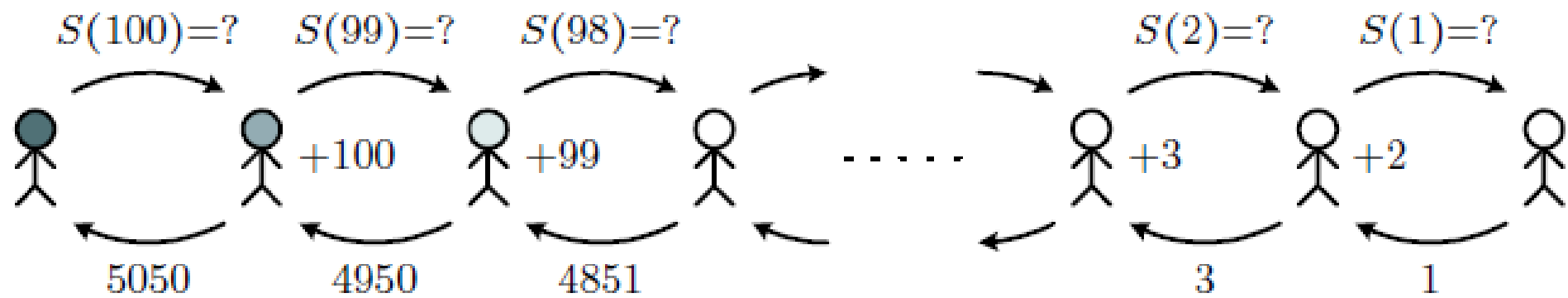
ฟังก์ชันเวียนเกิด (Recursive function)

- ฟังก์ชันที่เรียกตัวเองซ้ำไปซ้ำมาจนกว่าจะแก้ปัญหาได้
- แนวคิดในการแก้ปัญหา ด้วยการแบ่งปัญหานั้นออกเป็นปัญหาย่อย ๆ โดยที่
 - ปัญหาย่อยเหล่านั้นมีโครงสร้างเดียวกันกับปัญหาดั้งเดิม ซึ่งทำให้สามารถแก้ปัญหาได้ง่ายกว่า
 - เมื่อนำผลลัพธ์ของปัญหาย่อยเหล่านั้นมารวมกัน จะได้คำตอบของปัญหาดั้งเดิม
- อยู่บนพื้นฐานของการอุปนัย หรือการเหนี่ยวนำทางคณิตศาสตร์ (Mathematical Induction)





(a)

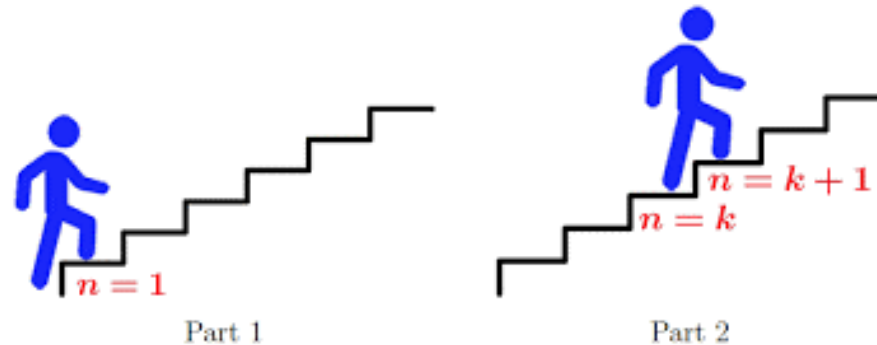


(b)

การเหนี่ยวนำทางคณิตศาสตร์ (Mathematical Induction)

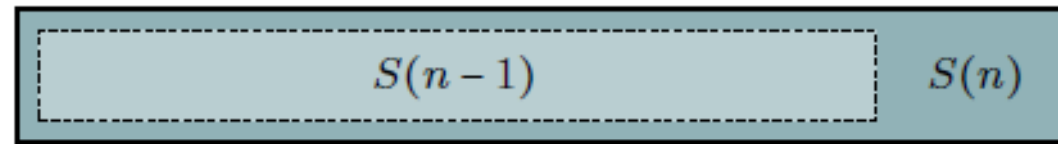
- การอุปนัย หรือ การเหนี่ยวนำทางคณิตศาสตร์ ใช้กับจำนวนเต็มที่มีมากกว่า 0

- Base case: ให้ $P(1)$ เป็นจริง

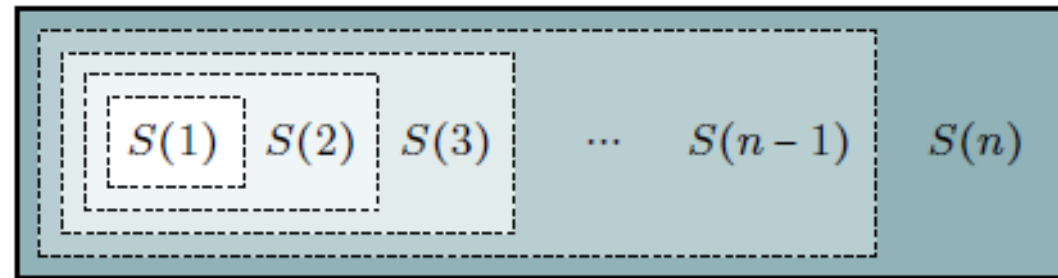


- Inductive case (Recursive case): ให้ $P(2) \dots P(k)$ เป็นจริง พิสูจน์ว่า $P(k+1)$ เป็นจริง



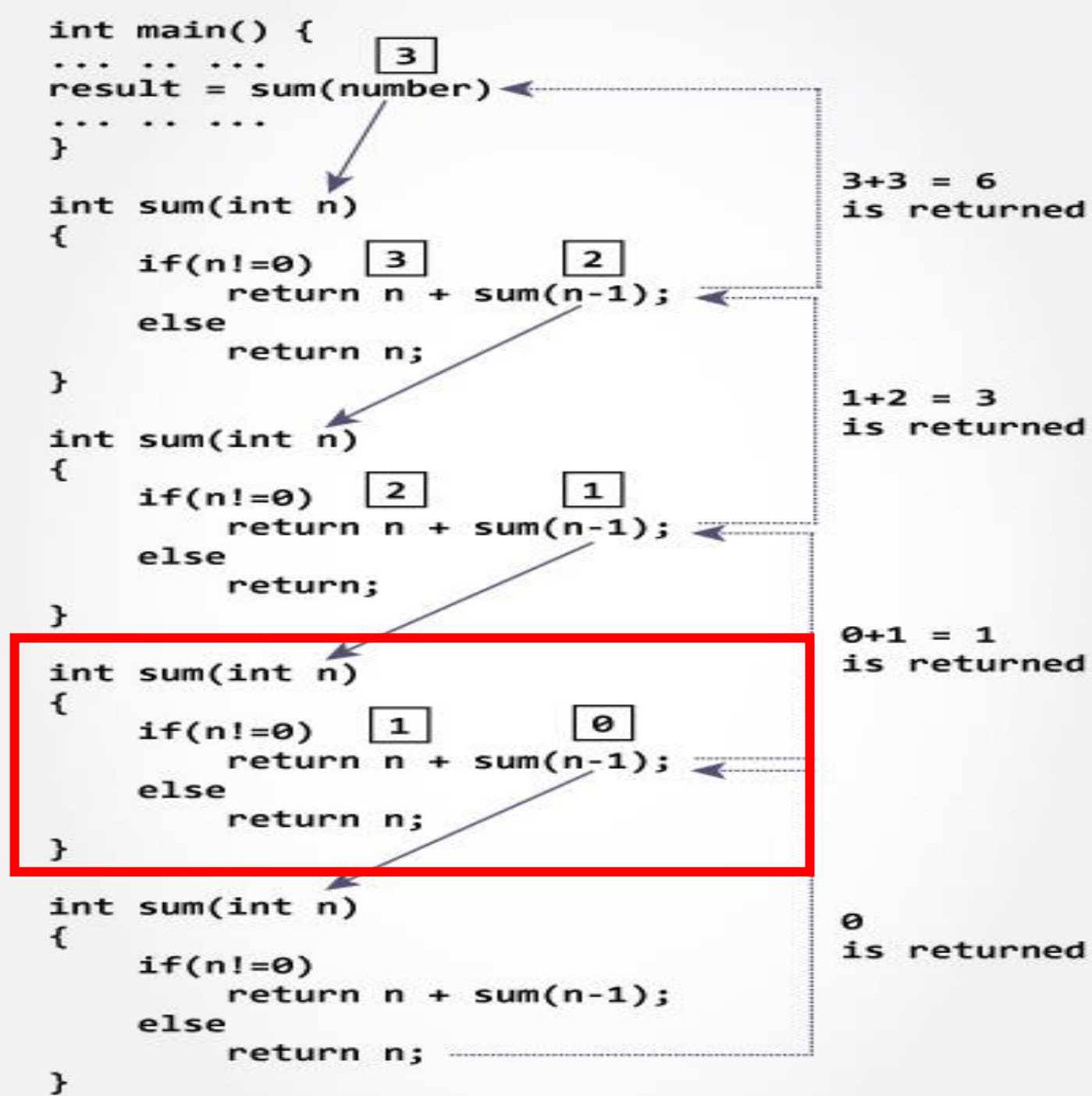


(a)



(b)





สรุปการทำงานของฟังก์ชันเวียนบังเกิด

- ฟังก์ชันหนึ่ง**เรียกใช้งานตัวเอง** โดยแต่ละครั้งที่ตัวมันเองถูกเรียก
- ปัญหาจะถูกแบ่งเป็น**ปัญหาย่อย (Recursive case)** ที่มี**ขนาดเล็กลง**
- และจะเกิดค่าตัวแปรหรือพารามิเตอร์**เปลี่ยนแปลง**ไปอย่างอัตโนมัติ
- แล้วกำหนด**การทำงานขั้นสุดท้าย (Base case)** ไว้ เมื่อทำงานถึงขั้นสุดท้ายก็จะสิ้นสุดการทำงานและส่งผลลัพธ์กลับไป

- เมื่อมีกรณีอย่างน้อย 1 กรณีที่สามารถหาคำตอบได้ทันที (base case หรือ simple case)
- ในขณะที่กรณีอื่นๆสามารถกำหนดได้ใหม่ (redefine) ในรูปที่ง่ายขึ้นกว่าเดิม เรียกว่า recursive case
- ปัญหาสามารถลดรูปลงมาสู่ simple case
 - *If this is a simple case*
solve it
 - else*
redefine the problem using recursion



เงื่อนไขในการหยุดการเวียนบังเกิด (Terminating condition)

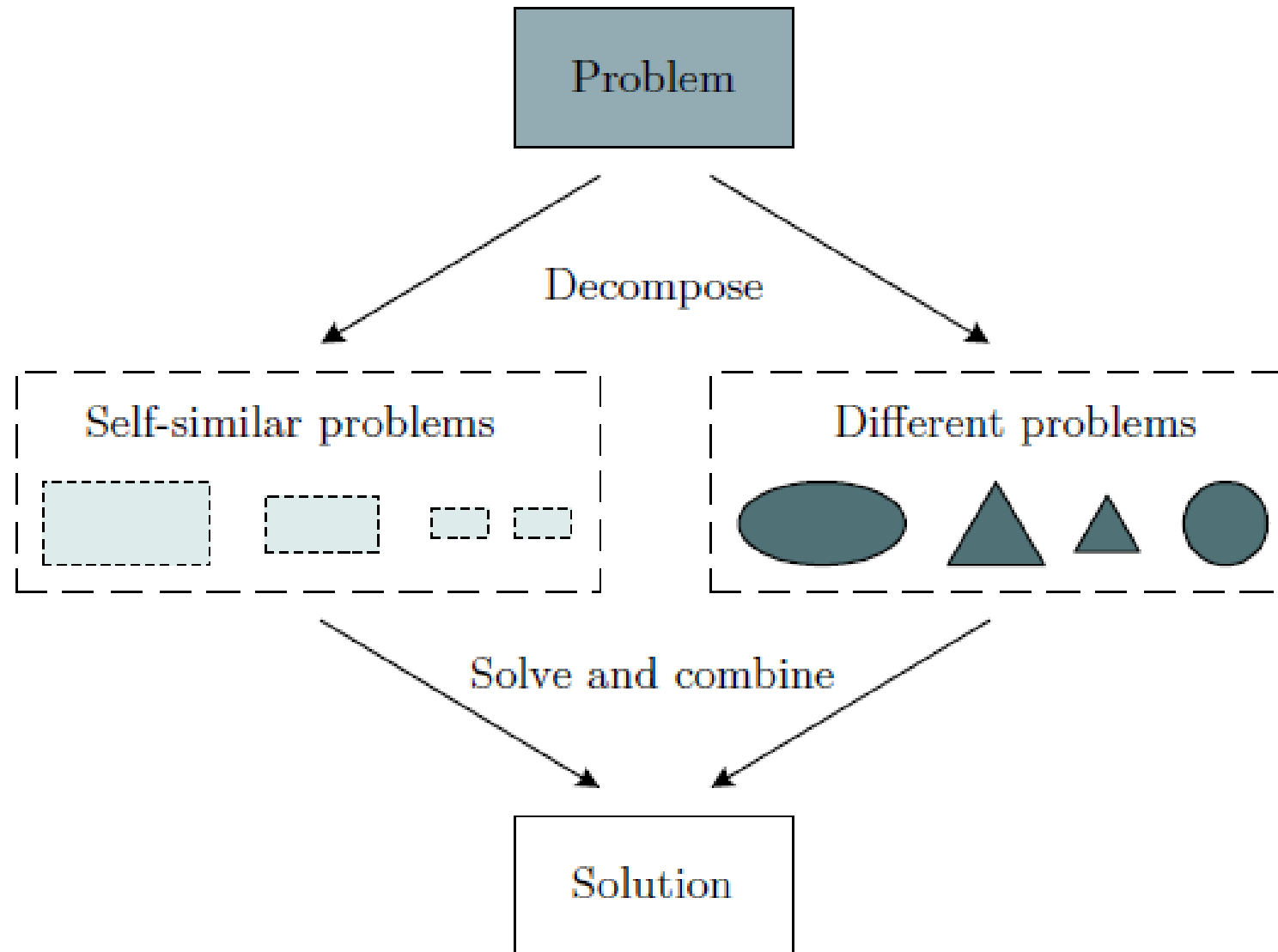
- recursive functions จะต้องมียุทธการการทำงานอย่างน้อย 1 ยุทธการเสมอ
 - โดยปกติ จะเป็นยุทธการของกรณีพื้นฐาน base case
- หากไม่มี terminating condition ฟังก์ชันอาจทำงานไม่หยุดเลย
 - เช่น ฟังก์ชันการคูณ if statement “if (n == 1) ...” คือ terminating condition.



การออกแบบฟังก์ชันเวียนบังเกิด

(Designing recursive function)

- 1) หาขนาดของปัญหา (n)
- 2) กำหนด Base case
- 3) แบ่งปัญหออกเป็นปัญหาย่อยที่มีโครงสร้างเหมือนปัญหาดั้งเดิม
- 4) กำหนด Recursive case ด้วยหลักการอุปนัย
- 5) ทดสอบโปรแกรม



ตัวอย่าง (Example)

```
#include <stdio.h>
int sum(int n);

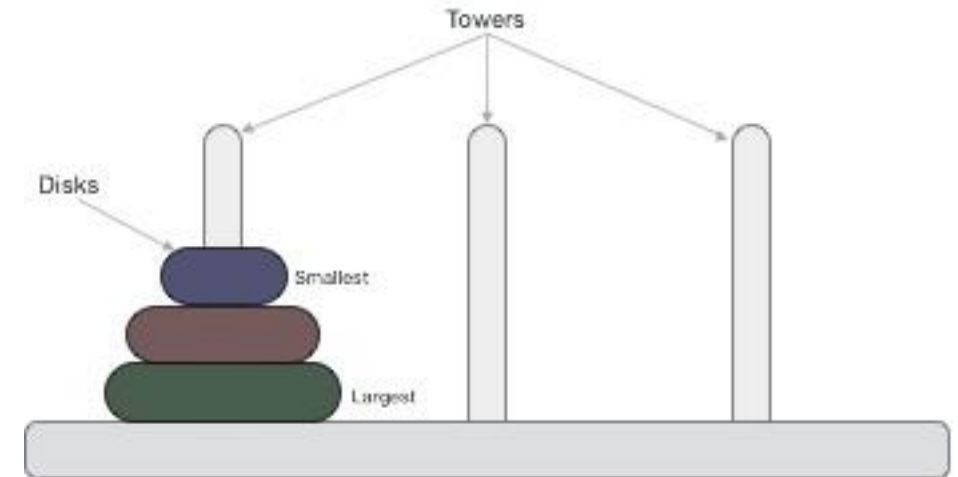
int main() {
    int number, result;

    printf("Enter a positive integer: ");
    scanf("%d", &number);
    result = sum(number);
    printf("sum = %d", result);
    return 0;
}
```

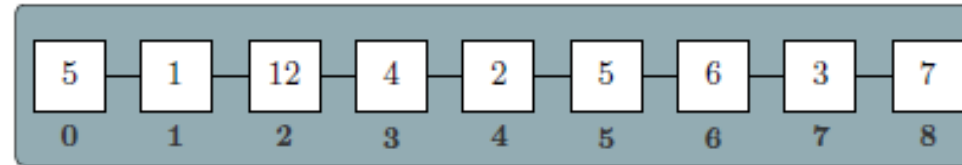
```
int sum(int num) {
    if (num!=0)
        return num + sum(num-1); // sum() function calls itself
    else
        return num;
}
```

รูปแบบของการเวียนเกิด (Types of recursion)

- Linear
 - Basic
 - Tail
- Multiple
 - Divine and Conquer
 - Puzzles and Fractal

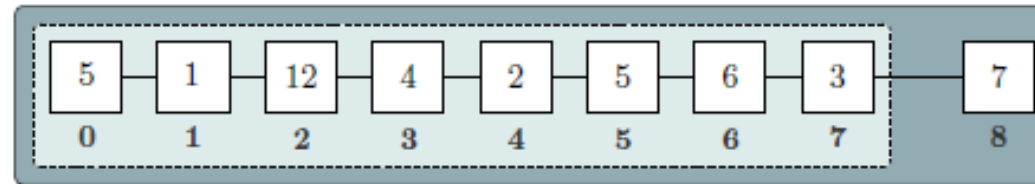


Original problem: $s(\mathbf{a}) = \mathbf{a}[0] + \mathbf{a}[1] + \dots + \mathbf{a}[n-1]$



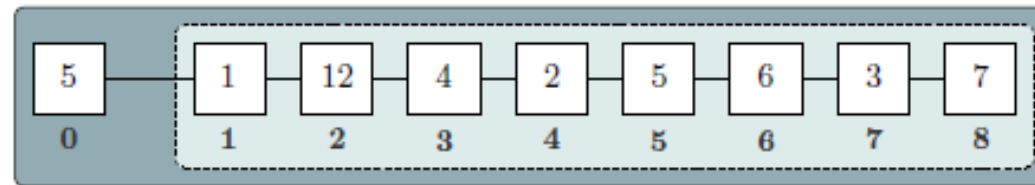
(a)

$$s(\mathbf{a}) = s(\mathbf{a}[0:n-1]) + \mathbf{a}[n-1]$$



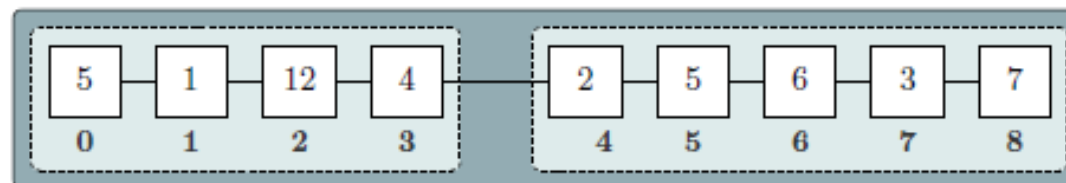
(b)

$$s(\mathbf{a}) = \mathbf{a}[0] + s(\mathbf{a}[1:n])$$



(c)

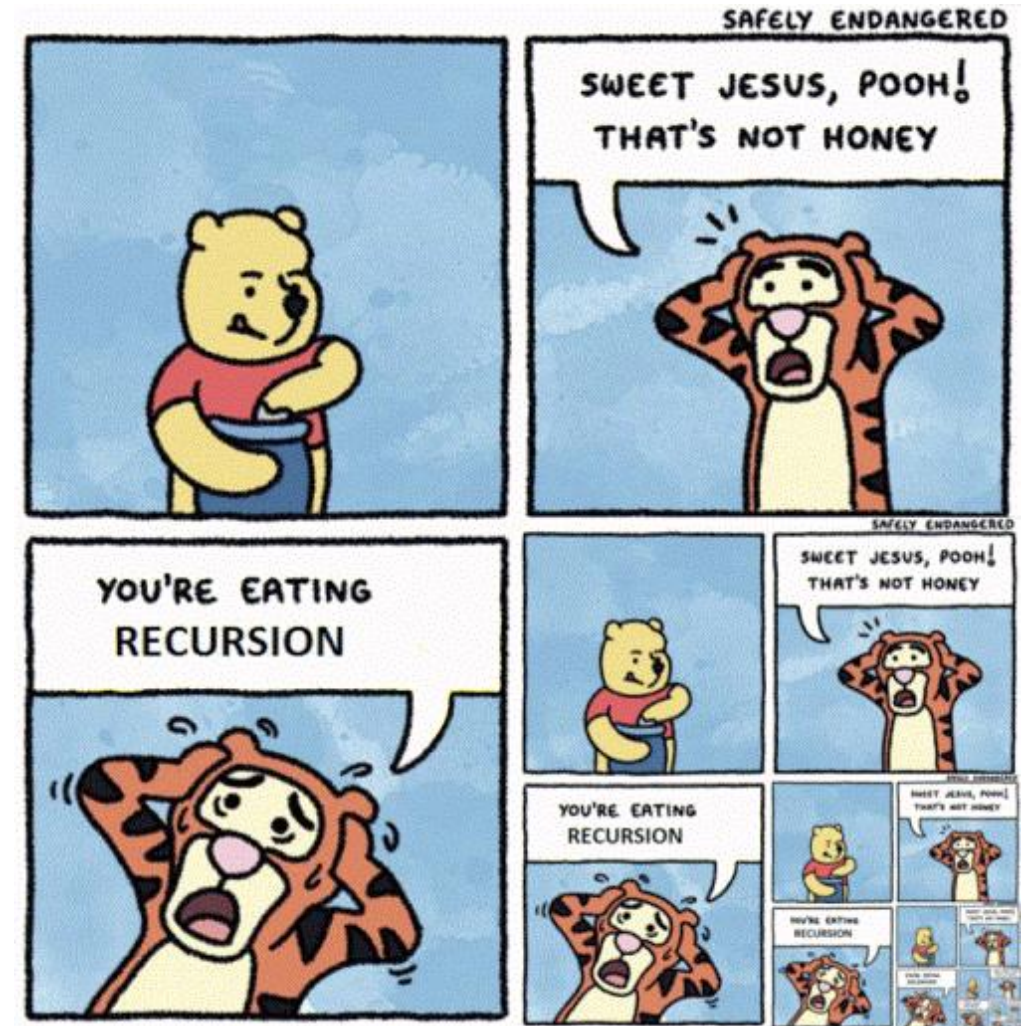
$$s(\mathbf{a}) = s(\mathbf{a}[0:n/2]) + s(\mathbf{a}[n/2:n])$$



(d)

การเวียนเกิด vs. การวนซ้ำ (Recursive vs. Iterative)

- การเวียนเกิด
 - เข้าใจง่ายกว่า
 - ออกแบบโปรแกรมง่ายกว่า
 - ใช้หน่วยความจำมากกว่า
 - ทำงานช้ากว่า



Lab

- เขียนโปรแกรมเพื่อหาค่าของการบวกเลขจำนวนเต็มตั้งแต่ 1 ถึง n , $n \leq 1000$
- เขียนโปรแกรมเพื่อหาค่าของพจน์ที่ n ในลำดับ Fibonacci ด้วยวิธีเวียนบังเกิด เมื่อ $n \leq 40$

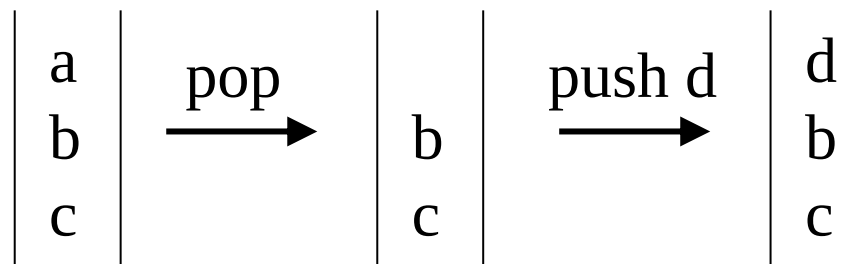


แบบฝึกหัดเขียนโปรแกรม (Programming exercises)

- เขียนโปรแกรมเพื่อหาค่า $n!$ ด้วยวิธีเวียนบังเกิด เมื่อ $n \leq 20$
 - เมื่อ $n! = n \cdot (n-1)!$ และ $0! = 1$
- เขียนโปรแกรมเพื่อคำนวณดอกเบี้ยทบต้น เมื่อมีเงินต้น b บาท อัตราดอกเบี้ย $r\%$ ต่อปี และเวลาผ่านไป n ปี ($0 < b < 10^6$, $r \leq 10$, $n \leq 100$)

How C Maintains the Recursive Steps

- ภาษา C เก็บรักษาค่าตัวแปรโดยใช้ **stack**
 - stack โครงสร้างข้อมูลที่จัดการข้อมูลในรูปแบบ LIFO
 - 2 operations คือ push และ pop



How C Maintains the Recursive Steps

- ทุกครั้งที่มีการเรียกฟังก์ชัน สถานะปัจจุบันของผู้เรียก (caller function) ได้แก่ parameters, local variables, and memory address เป็นต้น จะถูก push ลงใน stack
- เมื่อฟังก์ชันที่ถูกเรียกประมวลผลเสร็จ ค่าสถานะต่าง ๆ ของ caller function จะถูกนำกลับมาโดยการ pop จาก stack.

How to Trace Recursive Functions

- การ trace และ debug ฟังก์ชัน recursive ไม่ง่าย !!!
 - เช่น ถ้ามีการเรียกตัวเองนับร้อยครั้ง การ set break point หรือการ trace แบบ step-by-step จะน่าเหนื่อยมาก
- วิธีการที่นิยมคือ การเพิ่มคำสั่ง print เข้าไปเพื่อดูผลของการทำงานในแต่ละ recursive steps.

```
1.  /*
2.   * *** Includes calls to printf to trace execution ***
3.   * Performs integer multiplication using + operator
4.   * Pre:  m and n are defined and n > 0
5.   * Post: returns m * n
6.   */
7.  int
8.  multiply(int m, int n)
9.  {
10.     int ans;
11.
12.     printf("Entering multiply with m = %d, n = %d\n", m, n);
13.
14.     if (n == 1)
15.         ans = m;      /* simple case */
16.     else
17.         ans = m + multiply(m, n - 1); /* recursive step */
```

Watch the input arguments passed into each recursive step.

(continued)

สรุปเรื่องการเวียนเกิด

- โปรแกรมเข้าใจง่าย
- โปรแกรมมีขนาดสั้น จำนวนบรรทัดน้อย
- สามารถแก้ปัญหาบางประเภทได้อย่างรวดเร็ว
- มีลักษณะคล้ายวงวน (Repetition)
- หยุดทำงานเมื่อเงื่อนไขเป็นจริง
- ใช้หน่วยความจำเยอะ
- ทำงานช้ากว่าวงวน



การแบ่งแยกและเอาชนะ (Divide and Conquer)

- Divide - แบ่งปัญหาเป็นปัญหาย่อยที่มีโครงสร้างของปัญหาแบบเดียวกัน
- Conquer - แก้ปัญหาย่อยทั้งหมด
- Combine – นำคำตอบของปัญหาย่อยทั้งหมดมารวมกัน

การเวียนเกิด

孫子 SUN TZU's
ART of WAR

“

If his forces are
united, separate
them.

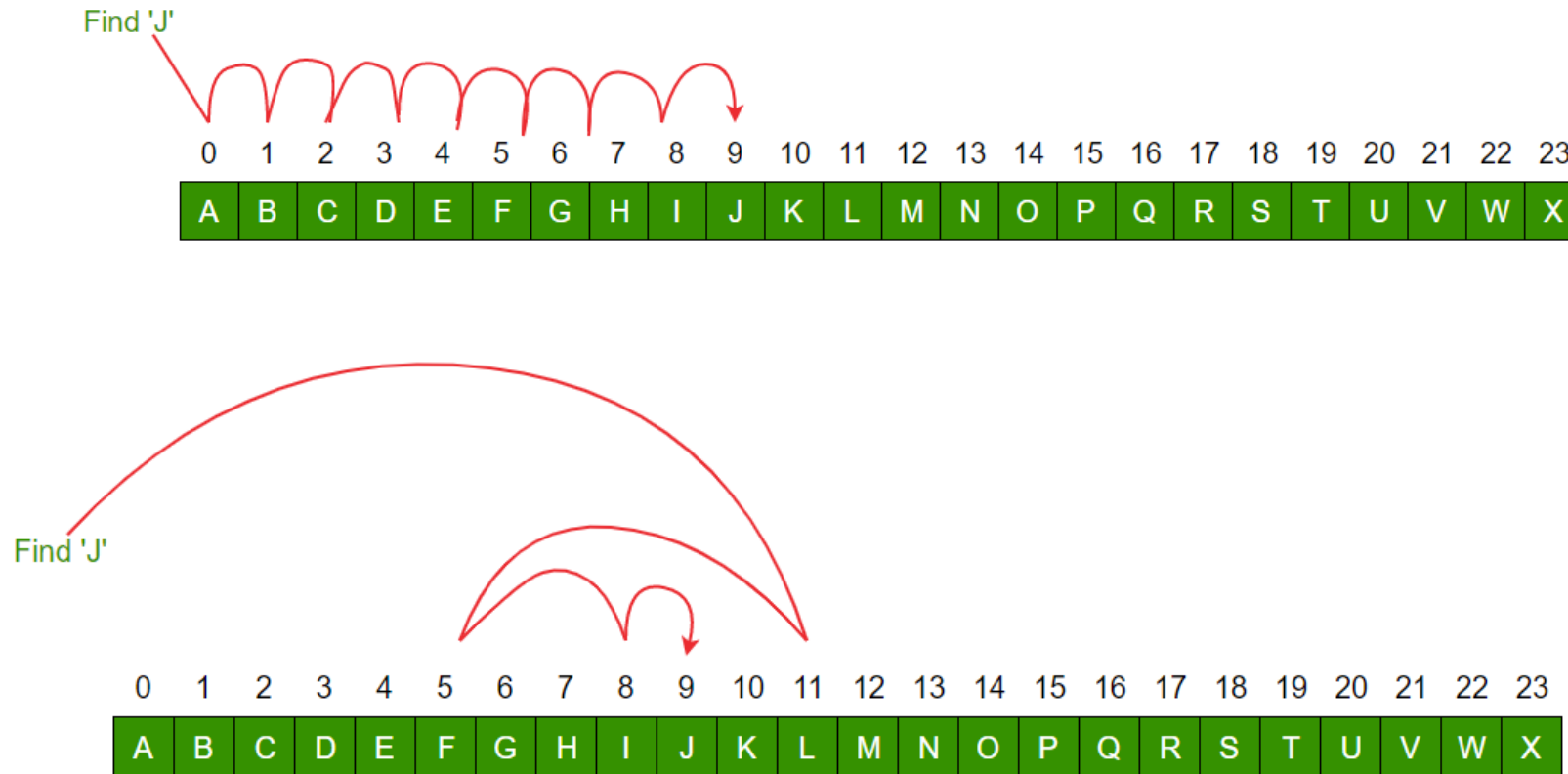
Sun Tzu

getnugget.co

ตัวอย่างปัญหา

- Binary search
- Merge sort
- Quick sort
- Integer multiplication
- Matrix multiplication
- Maxima set
- Counting inversion

Binary search

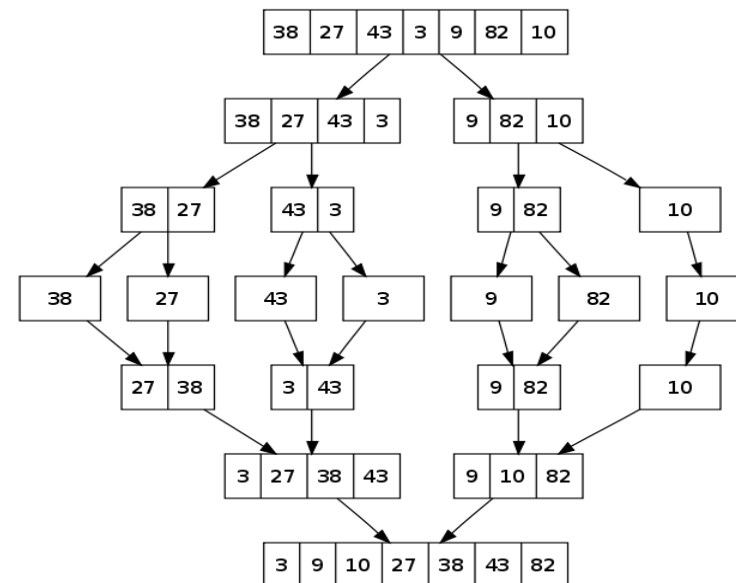
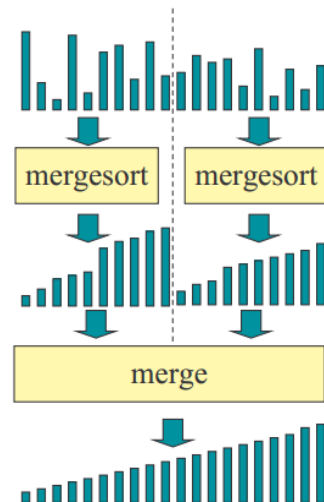


- เขียนโปรแกรมเพื่อค้นหาจำนวนเต็ม x ในอาร์เรย์ของจำนวนเต็มขนาด n ด้วยวิธีการ Binary search ด้วยวิธีเวียนบังเกิด $n \leq 1000$

Merge Sort

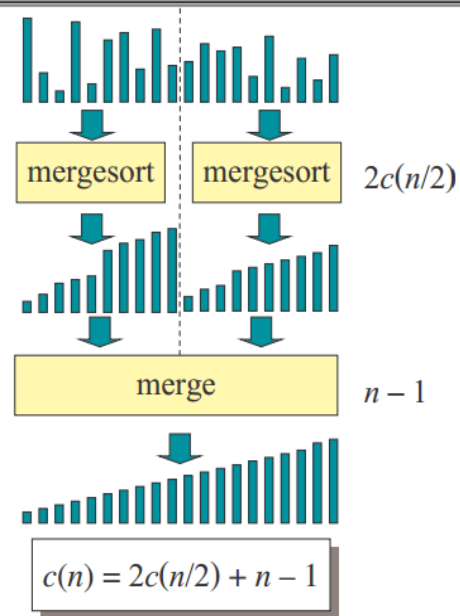
- การเรียงลำดับแบบผสาน โดยการแบ่งแถวลำดับออกเป็น 2 ชุดย่อย ชุดทางซ้ายและชุดทางขวาวขนาดพอๆ กัน
- นำข้อมูลทั้ง 2 ชุดมาเรียงลำดับ
- นำผลของสิ่งที่ได้มาผสานกัน ผลลัพธ์สุดท้ายจะเป็นผลลัพธ์ที่เรียงลำดับ

Merge Sort : Divide & Conquer



Merge sort analysis

Merge Sort : Analysis



Merge Sort : Analysis

$$\begin{aligned}
 c(n) &= 2c(n/2) + (n-1) \quad n > 1, c(0) = c(1) = 0, \quad n = 2^k \\
 &= 2\left(2c(n/2^2) + (n/2 - 1)\right) + (n-1) \\
 &= 2^2 c(n/2^2) + (n-2) + (n-1) \\
 &= 2^3 c(n/2^3) + (n-4) + (n-2) + (n-1) \\
 &\dots \\
 &= 2^k c(n/2^k) + kn - (2^{k-1} + 2^{k-2} + \dots + 2^0) \\
 &= n \log n - n + 1 \\
 &= O(n \log n)
 \end{aligned}$$

```
void merge(int A[], int p, int q, int r)
{
    /* Create L ← A[p..q] and M ←
    A[q+1..r] */
    int n1 = q - p + 1;
    int n2 = r - q;

    int L[n1], M[n2];
    for (i = 0; i < n1; i++)
        L[i] = A[p + i];
    for (j = 0; j < n2; j++)
        M[j] = A[q + 1 + j];

    /* Maintain current index of sub-
    arrays and main array */
    int i, j, k;
    i = 0;
    j = 0;
    k = p;
```

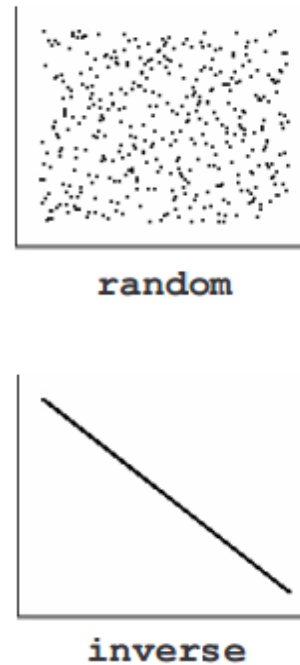
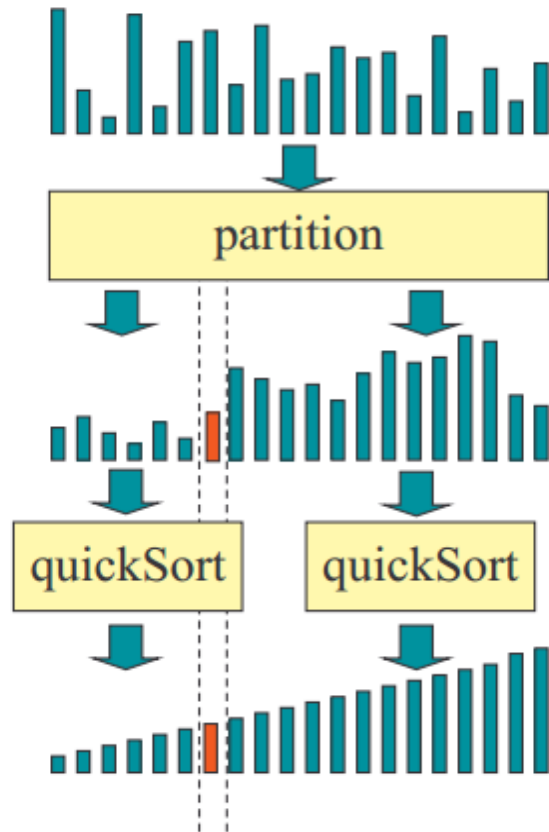
```
/* Until we reach either end of either
L or M, pick larger among elements L
and M and place them in the correct
position at A[p..r] */
while (i < n1 && j < n2)
{
    if (L[i] <= M[j])
    {
        arr[k] = L[i];
        i++;
    }
    else
    {
        arr[k] = M[j];
        j++;
    }
    k++;
}
```

```
/* When we run out of elements in
either L or M, pick up the remaining
elements and put in A[p..r] */
while (i < n1)
{
    A[k] = L[i];
    i++;
    k++;
}

while (j < n2)
{
    A[k] = M[j];
    j++;
    k++;
}
}
```

Quick sort

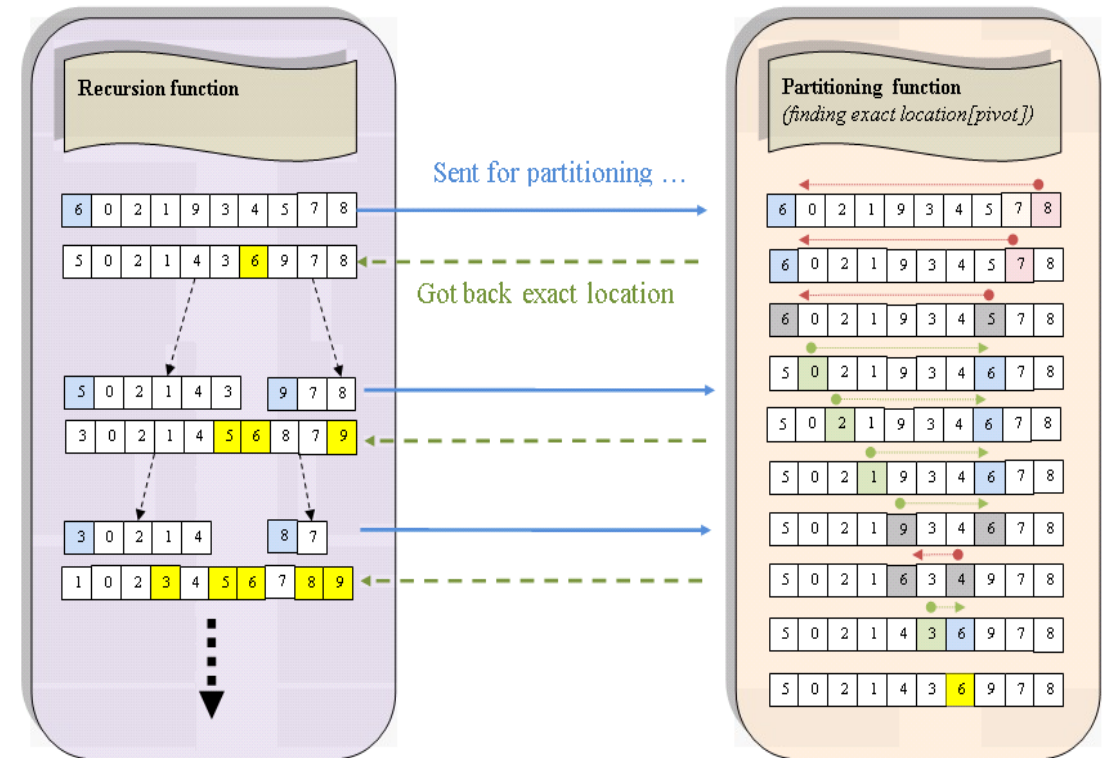
Quick Sort (C. A. R. Hoare, 1961)



Quick Sort

Legend:

- pointer
- Looking For SMALLER value
- swap
- Looking For GREATER value



Quick sort

```
void swap(int *a, int *b)
{
    int t = *a;
    *a = *b;
    *b = t;
}
```

```
int partition(int array[], int low, int high)
{
    int pivot = array[high];
    int i = (low - 1);
    for (int j = low; j < high; j++)
    {
        if (array[j] <= pivot)
        {
            i++;
            swap(&array[i], &array[j]);
        }
    }
    printArray(array, 7);
    cout << ".....\n";
    swap(&array[i + 1], &array[high]);
    return (i + 1);
}
```

```
void quickSort(int array[], int low, int high)
{
    if (low < high)
    {
        int pi = partition(array, low, high);
        quickSort(array, low, pi - 1);
        quickSort(array, pi + 1, high);
    }
}
```

Integer multiplication with Brute force

ในการคำนวณตัวเลข การคูณ (multiplication) และการหาร (division) ถือเป็นงานที่ต้องใช้ทรัพยากรในการคำนวณสูงเมื่อเปรียบเทียบกับ การบวก (addition) และการลบ (subtraction) โดยเฉพาะอย่างยิ่งหากตัวเลขมีจำนวนหลายๆ หลัก

วิธีการคูณเลข 2 จำนวน ที่เรารู้กันเคยมากที่สุด คือ *pen-and-pencil algorithm* โดยเริ่มจากการพิจารณานำตัวคูณในแต่ละหลักไปคูณหลักของตัวตั้ง จากนั้นเลื่อนตัวคูณไปหลักถัดไปและทำซ้ำจนกระทั่งครบทุกหลัก และนำมาผลลัพธ์บวกกัน

$$\begin{array}{r}
 23958233 \\
 \times \quad 5830 \\
 \hline
 00000000 \\
 71874699 \\
 191665864 \\
 + 119791165 \\
 \hline
 139676498390
 \end{array}$$

จากตัวอย่างจะเห็นได้ว่า หากตัวตั้งและตัวคูณมีจำนวนหลักเท่ากันคือ n

จำนวนครั้งในการคูณทั้งหมดคือ n^2

Integer multiplication with divide & conquer

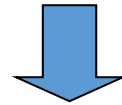
เพื่อลดจำนวนครั้งในการคูณเราอาจใช้เทคนิคการแบ่งแยกและเอาชนะสำหรับแก้ปัญหานี้ได้ แต่เราจะต้องหาวิธีในการแบ่งปัญหาและการรวมปัญหาให้ได้ ซึ่งในปัญหานี้อาจกระทำได้ง่าย ดังนี้

สมมติให้เลขจำนวนเต็ม 2 หลักคือ 23 และ 14 เราสามารถเขียนในอีกรูปแบบดังนี้

$$23 = 2 \cdot 10^1 + 3 \cdot 10^0$$

การแบ่งปัญหาย่อย

$$14 = 1 \cdot 10^1 + 4 \cdot 10^0$$



$$23 * 14 = (2 \cdot 10^1 + 3 \cdot 10^0) * (1 \cdot 10^1 + 4 \cdot 10^0) \quad \text{การรวมคำตอบ}$$

$$23 * 14 = (2 * 1)10^2 + (2 * 4 + 3 * 1)10^1 + (3 * 4)10^0 = 322$$

Integer multiplication with divide & conquer

ดังนั้นเลขจำนวนเต็ม 2 ตัว คือ X และ Y ขนาด n หลัก สามารถแบ่งปัญหาย่อยได้ครั้งละ 4 เทอม คือ a, b, c และ d สำหรับตัวตั้งและตัวคูณ ดังนี้



$$x = a.10^{n/2} + b$$



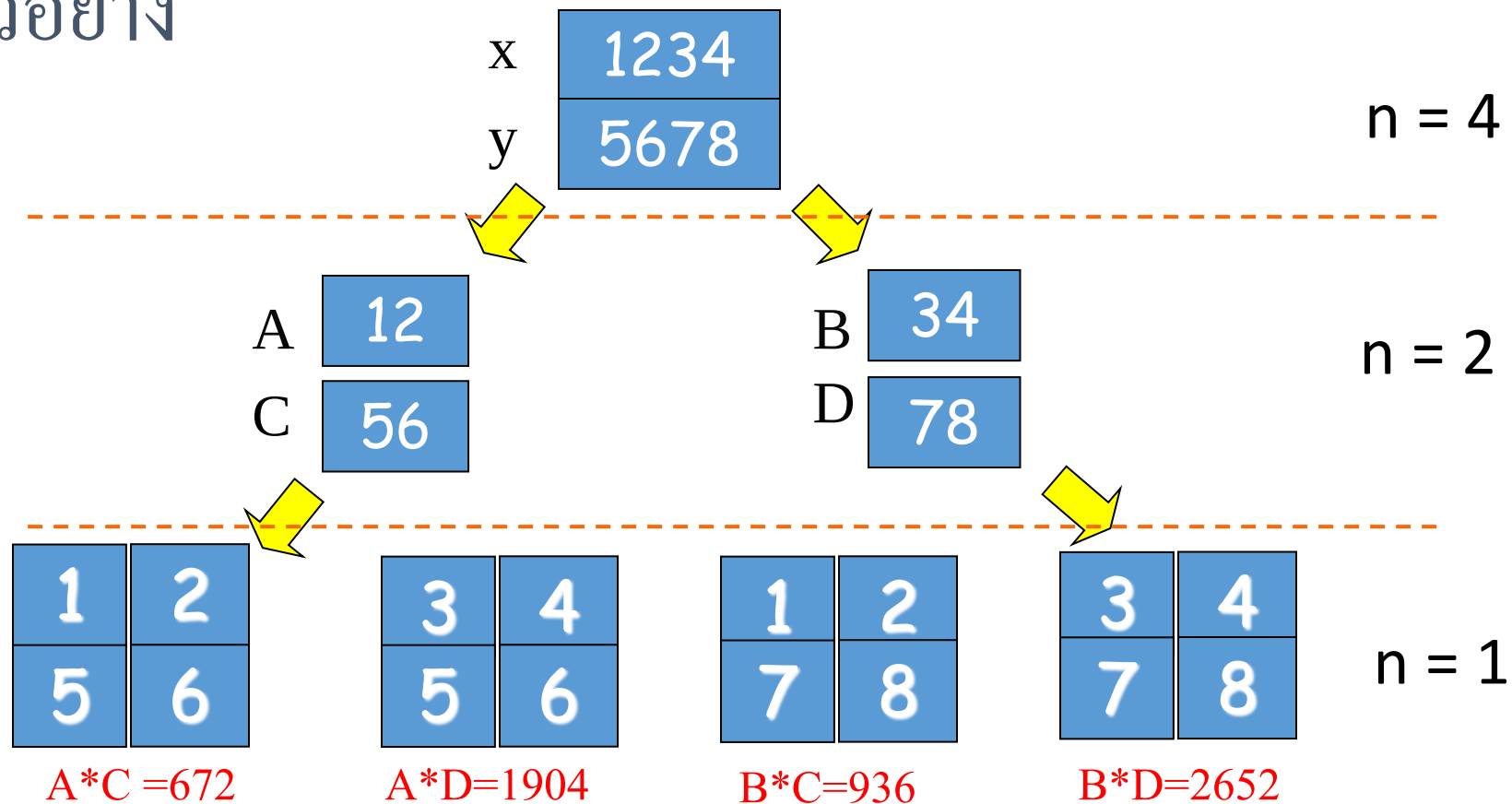
$$y = c.10^{n/2} + d$$

และการรวมคำตอบของปัญหาย่อยคือ

$$x * y = (a.10^{n/2} + b) * (c.10^{n/2} + d)$$

$$x * y = (a * c).10^n + (a * d + b * c).10^{n/2} + b * d$$

ตัวอย่าง



$$\begin{aligned}
 1234 * 5678 &= (a*c)10^n + (a*d + b*c)10^{n/2} + (b*d) \\
 &= 672*10^4 + (1,904+936)*10^2 + (2,652) \\
 &= 7,006,652
 \end{aligned}$$

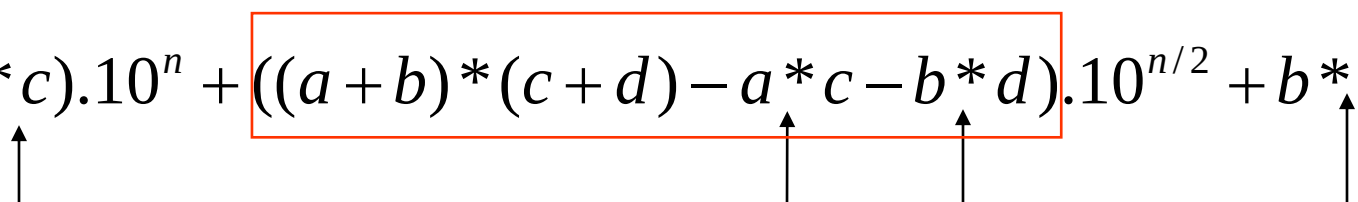
Karatsuba method (1962)

จากหลักการ divide & conquer จะเห็นได้ว่าจำนวนครั้งในการคูณไม่ได้ลดลงไปจากวิธีการ brute force คือ n^2

ต่อมาในปี ค.ศ. 1962 Anatoly Karatsuba จะได้นำเสนอการแยกตัวประกอบในเทอมกลางเพื่อลดจำนวนครั้งในการคูณดังนี้

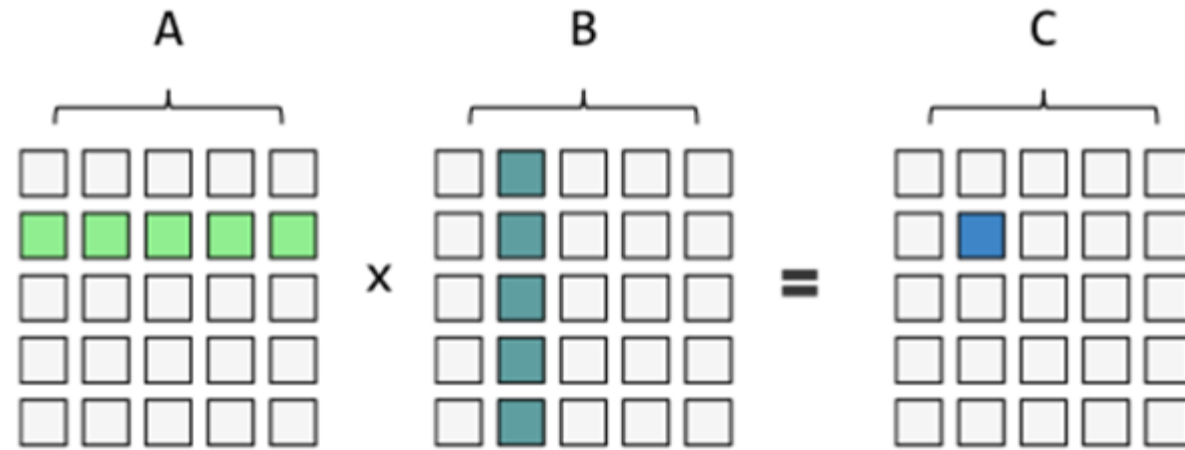
$$x * y = (a * c).10^n + (a * d + b * c).10^{n/2} + b * d$$

โดยพยายามแยกตัวประกอบในเทอมกลางเพื่อให้เกิดเทอมที่ซ้ำกันออกมา

$$x * y = (a * c).10^n + ((a + b) * (c + d) - a * c - b * d).10^{n/2} + b * d$$


จะเห็นว่าด้วยวิธีการแยกตัวประกอบนี้ จะทำให้สามารถลดจำนวนครั้งในการคูณกัน จาก 4 ครั้งในแต่ละรอบของการแบ่งเหลือเพียง 3 ครั้งเท่านั้น

Matrix multiplication



Brute force algorithm

```
void multiply (int A[][N], int B[][N], int C[][N])
{
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            { C[i][j] = 0;
              for (int k = 0; k < N; k++)
                  C[i][j] += A[i][k]*B[k][j];
            }
}
```

Matrix multiplication with divide and conquer

แบ่งเมตริกซ์ A และ B ที่ละ $n/2$ จนกระทั่งเมตริกซ์ A และ B มีขนาด 2×2

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \times \begin{bmatrix} e & f \\ g & h \end{bmatrix} = \begin{bmatrix} ae + bg & af + bh \\ ce + dg & cf + dh \end{bmatrix}$$

A B C

A, B and C are square matrices of size $N \times N$

a, b, c and d are submatrices of A, of size $N/2 \times N/2$

e, f, g and h are submatrices of B, of size $N/2 \times N/2$

หากขนาดของเมตริกซ์ A และ B คือ $n \times n$ จำนวนครั้งในการคูณ สำหรับเมตริกซ์ C คือ n^3

Strassen's Matrix Multiplication (1969)

ลดจำนวนครั้งในการคูณเมตริกซ์ C เหลือเพียง 7 ครั้ง จากปกติ 8 ครั้ง แต่เพิ่มจำนวนครั้งของการบวกและลบ เป็น 18 ครั้ง กรณีที่เมตริกซ์ A และ B มีขนาด 2×2

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \times \begin{bmatrix} e & f \\ g & h \end{bmatrix} = \begin{bmatrix} p5 + p4 - p2 + p6 & p1 + p2 \\ p3 + p4 & p1 + p5 - p3 - p7 \end{bmatrix}$$

A B C

$$p1 = a(f - h)$$

$$p3 = (c + d)e$$

$$p5 = (a + d)(e + h)$$

$$p7 = (a - c)(e + f)$$

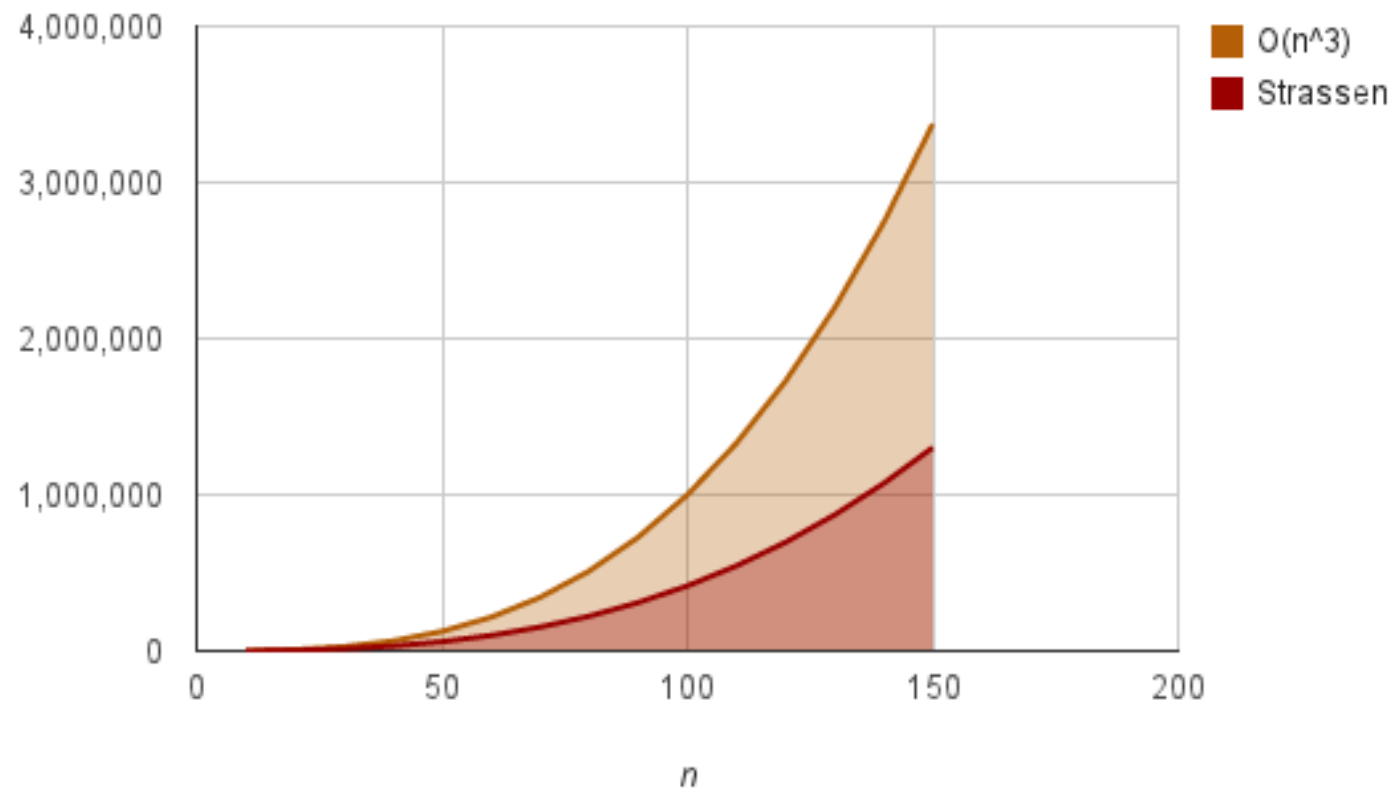
$$p2 = (a + b)h$$

$$p4 = d(g - e)$$

$$p6 = (b - d)(g + h)$$

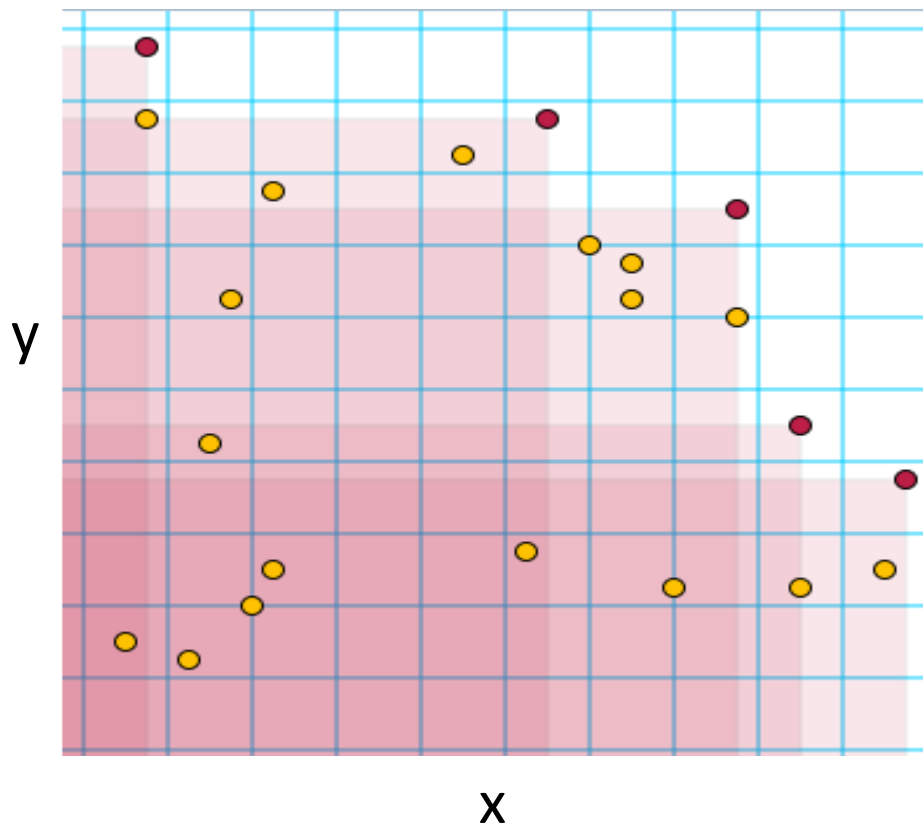
Strassen's Matrix Multiplication

เปรียบเทียบจำนวนครั้งในการคูณเมื่อ n เพิ่มขึ้น



Maxima set problem

ในระนาบ X,Y กำหนดให้มี n จุด จุด p คือ **maximal** หากไม่มีจุด q ใดๆ ที่สามารถ
ครอบคลุม (dominated) p ได้
หมายเหตุ p จะไม่ถูกรวมคลุมโดย q หาก $(p.x > q.x)$ และ $(p.y > q.y)$

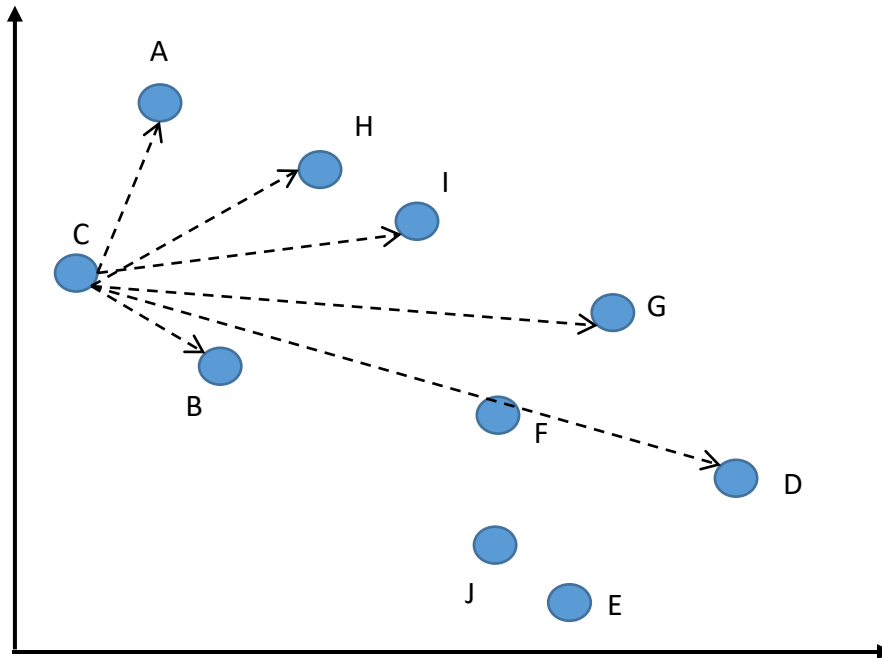


ดังนั้นปัญหานี้ก็คือการออกแบบอัลกอริทึม
ที่มีประสิทธิภาพในการค้นหาเซตของ
maximum points

Maxima set problem with Brute force

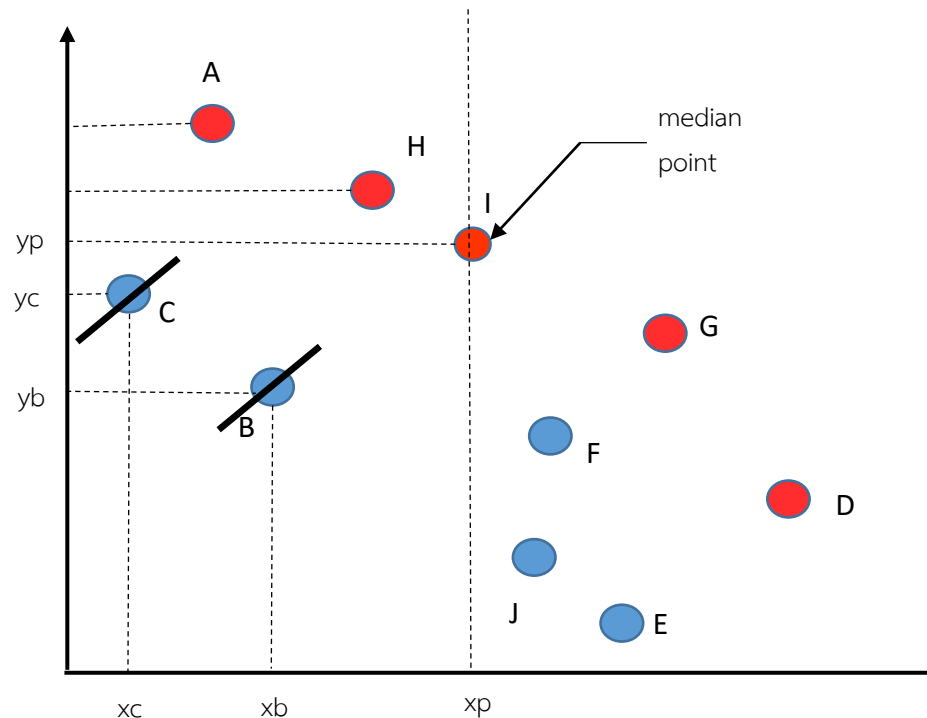
หลักการ brute force ก็คือไล่เช็คทีละจุด โดยหากพบว่ามีอย่างน้อย 1 จุดที่ถูกครอบคลุมโดยจุดใดๆ ก็จะถือว่าจุดดังกล่าวไม่ได้อยู่ในเซตคำตอบ

ดังนั้นหากมีทั้งหมด n จุด คาดหวังได้ว่าจำนวนจุดที่ต้องเปรียบเทียบทั้งหมดคือ n^2



Divide & Conquer Solution

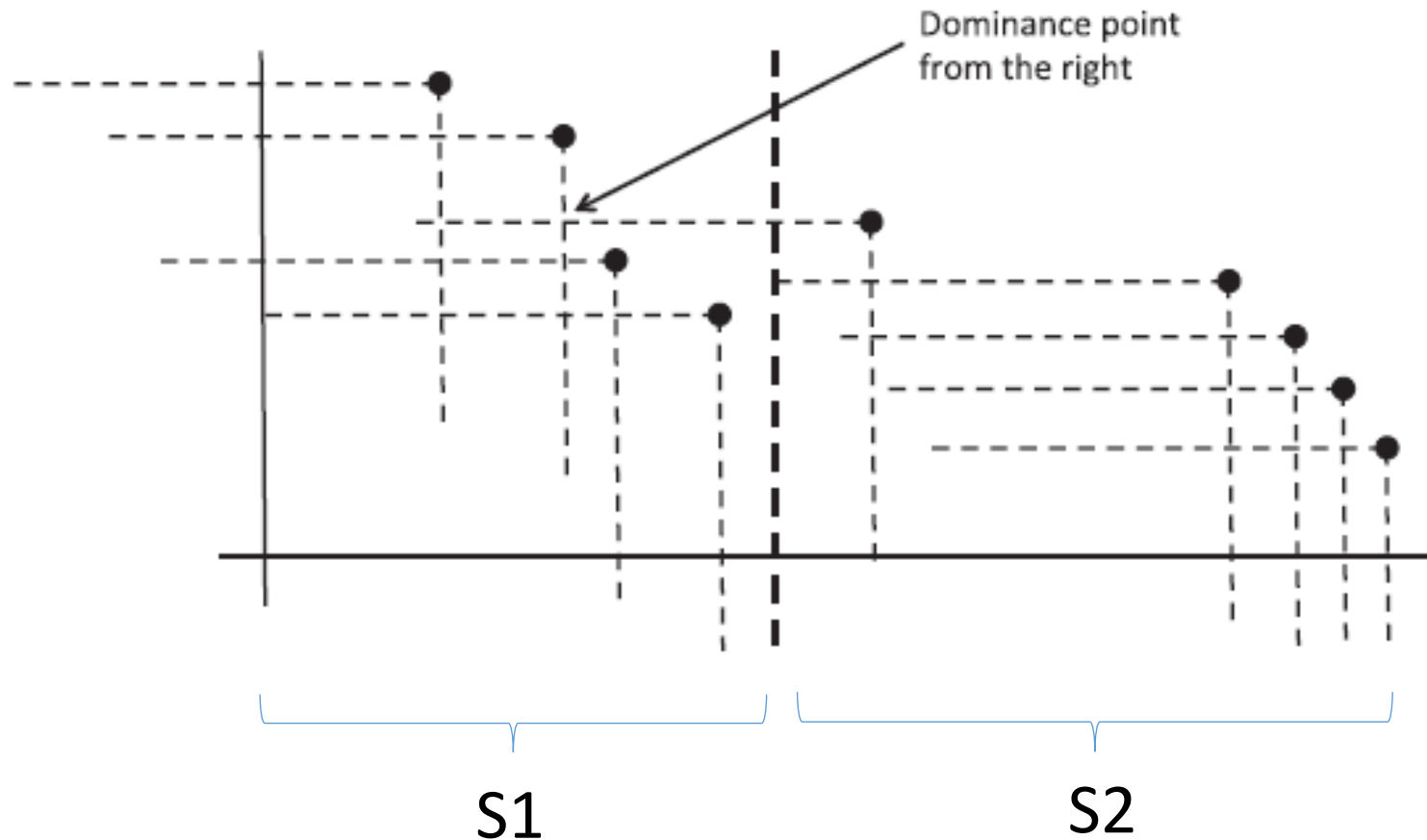
1. เรียงลำดับจุดทั้งหมดสอดคล้องสัมพันธ์กับค่า x (lexicographic order)
2. ค้นหาจุดกลาง (median point) เป็นจุดอ้างอิงและเป็นตัวแบ่งกลุ่มออกเป็น 2 กลุ่ม
3. ทุกๆ จุดด้านซ้ายที่มีค่า y มากกว่า y_p คือ maxima point
4. ทุกๆ จุดด้านขวาเป็น maxima point ยกเว้นแต่จะมีบางจุดด้านขวามีค่า y มากกว่า



Divide & Conquer Solution

อย่างไรก็ตามอาจมีบางจุดที่เป็น maxima ด้านซ้าย S1 ถูกครอบคลุมโดยบางจุดที่เป็น maxima ด้านขวา S2

วิธีการแก้ปัญหาก็คือทุกครั้งที่มีการ add จุดเข้าไปที่ S2 จะต้องนำค่า y ของจุดนั้นไปเช็กับทุกๆ จุด maxima ใน S1 และกำจัดทิ้งหากพบว่าค่า y ของจุดใน S1 มีค่าน้อยกว่า



Divide & Conquer Solution

ประมวลผลเบื้องต้น เรียงลำดับจุดทั้งหมดสอดคล้องสัมพันธ์กับค่า x (lexicographic order)

Base case: ถ้า $n = 1$ เพิ่มจุดเข้าไปในเซตคำตอบ S

Divide: ให้ p เป็นจุดกลาง (median) ดังนั้นแบ่งจุดที่เหลือออกเป็น 2 กลุ่มคือ $S1$ และ $S2$
โดยทุกๆ จุดใน $S1$ จะมีค่า x น้อยกว่า x_p มิฉะนั้นจะถูกเก็บไว้ใน $S2$

Combine:

** ให้ q เป็นจุดที่มีค่า x ต่ำสุดใน $S2$ จากนั้นกำจัดทุกๆ จุด ใน $S1$ ที่มีค่า y น้อยกว่า y_q
เซตคำตอบคือ $S = S1 \cup S2$

Solution

Algorithm MaximaSet(S):

Input: A set, S , of n points in the plane

Output: The set, M , of maxima points in S

if $n \leq 1$ **then**

return S

Let p be the median point in S , by lexicographic (x, y) -coordinates

Let L be the set of points lexicographically less than p in S

Let G be the set of points lexicographically greater than or equal to p in S

$M_1 \leftarrow \text{MaximaSet}(L)$

$M_2 \leftarrow \text{MaximaSet}(G)$

Let q be the lexicographically smallest point in M_2

for each point, r , in M_1 **do**

if ~~$x(r) \leq x(q)$~~ **and** $y(r) \leq y(q)$ **then**

 Remove r from M_1

return $M_1 \cup M_2$

Counting Inversion

- Given $A[n] = \{a_1, a_2, \dots, a_n\}$

- Inversions i and j

- if $i < j$ but $a_i > a_j$

- Example

$A = \{1, 3, 4, 2, 5\}$

There are 2 inversions = 3-2, 4-2

- Brute force: check all $O(n^2)$ pairs i and j

Counting Inversion: Divide and Conquer algorithm

1. Divide: separate list into two pieces.
2. Conquer: recursively count inversions in each half.
3. **Combine: count inversions where a_i and a_j in different halves**

1	5	4	8	10	2	6	9	12	11	3	7
---	---	---	---	----	---	---	---	----	----	---	---

$O(1)$

1	5	4	8	10	2	6	9	12	11	3	7
---	---	---	---	----	---	---	---	----	----	---	---

$2T(N / 2)$

5 blue-blue inversions 8 green-green inversions

How to Co